

BUET M.Sc. in CSE Admission Test

Group 1 Preparation Guide

10 syllabus areas | 100 MCQs with explanations | 53 written questions with model answers |
3-day plan | past-paper pattern index

How to use this guide

1. Read the key-facts box of a topic (5 minutes).
2. Attempt the MCQs and written questions **on paper before looking at the answers**.
3. Mark misses, re-do them the next day.
4. Appendix A lists the themes seen in past BUET papers; use it as a final checklist.

Honest note: all questions and answers here are written by Claude in the style of past papers; they are not official BUET questions. Verify code outputs and numerical results against your textbooks. Formats vary by year, so check your current admission circular.

Contents

Contents	2
Read This First: Exam Format, Strategy and 3-Day Plan	3
Topic 1: Structured and Object-Oriented Programming (C / C++ / Java)	4
Topic 2: Discrete and Concrete Mathematics, Probability and Statistics	8
Topic 3: Data Structures and Algorithms	11
Topic 4: Compiler Design and Theory of Computation	15
Topic 5: Database Systems	19
Topic 6: Computer Networks and Security	23
Topic 7: Software Engineering and Information System Design	27
Topic 8: Digital Logic Design, Microprocessors and Microcontrollers	31
Topic 9: Computer Architecture and Operating Systems	35
Topic 10: Artificial Intelligence and Machine Learning	39
Appendix A: Past-Paper Pattern Index	43
Appendix B: One-Page Formula Sheet	43
Appendix C: Sources and Where to Practise More	43

Read This First: Exam Format, Strategy and 3-Day Plan

Format (from the admission circular you shared)

Part	Details
MCQ	50 questions, 50 marks, 50 minutes, 0.5 marks deducted per wrong answer
Written	10 questions, 100 marks, 1 hour (about 10 marks each)
Rules	The written script is evaluated only if the MCQ score is at or above the committee cut-off. Final merit = MCQ + written. Only non-programmable calculators are allowed.

Past sessions used other formats (a 2019 paper had 30 questions of 5 marks in 1.5 hours; a 2021 online session had 100 MCQs; 2024 had 30 questions of 10 marks). Confirm the numbers in your own circular, but the topic list is stable.

MCQ guessing rule (0.5 negative, 4 options, 1 mark each)

Options you can eliminate	Chance right	Expected marks
0	25%	-0.125 (skip)
1	33%	0 (neutral)
2	50%	+0.25 (guess)

Guess only when you can eliminate at least two options. Do the sure MCQs first, then return to the doubtful ones.

Written-exam strategy

- Read all 10 questions first; start with your strongest three to bank marks and confidence.
- Budget about 6 minutes per question. Write the definition or formula first, then a small example or diagram: partial marks are common.
- For code: write clean, short C/C++/Java, state complexity. For diagrams (class, state, automata): draw neatly and label transitions.
- For "explain/differentiate" questions use a small table or bullet points; examiners scan for key terms.
- Non-programmable calculator: practise mod, powers and logs by hand; know $\log_2$ of 3, 5, 7 approximations.

3-day plan

Day	Topics	Evening check
Day 1	Programming (1), Discrete/Probability/Statistics (2), DSA (3), Compiler and Theory of Computation (4). These four have the most repeated question patterns.	Redo all missed MCQs of topics 1-4; write out two DFA/LL(1) problems from memory.
Day 2	Database (5), Networks and Security (6), Software Engineering (7), AI and ML (10).	Redo misses of Day 1 and 2; do 5 subnetting problems; write one SQL query per clause type.
Day 3	Digital Logic and Microprocessors (8), Architecture and OS (9). Then a timed mock: 50 MCQs in 50 min (mix 5 per topic from this guide), then 10 written questions in 60 min.	Skim the Past-Paper Pattern Index (Appendix A) and formula sheet (Appendix B). Prepare admit card and calculator. Sleep early.

Topic 1: Structured and Object-Oriented Programming (C / C++ / Java)

Key facts to revise

- **Conversions:** in a mixed signed/unsigned comparison the signed value becomes unsigned. Integer division truncates ($7/2 = 3$). char/short promote to int in expressions.
- **Pointers:** $a[i]$ is $*(a+i)$; $p+1$ moves by $\text{sizeof}(*p)$ bytes. An array name decays to a pointer (sizeof of a pointer parameter is the pointer size, not the array size).
- **Undefined behaviour (UB):** modifying a variable twice without a sequence point ($a++ + ++a$), signed overflow, out-of-bounds access, dereferencing an uninitialised/dangling pointer, writing to a string literal, returning the address of a local variable.
- **Storage:** a static local keeps its value between calls; globals/statics are zero-initialised; locals are not. sizeof is evaluated at compile time (its operand is not executed).
- **OOP:** encapsulation, inheritance, polymorphism, abstraction. Overloading = compile-time (same name, different parameters). Overriding = run-time (same signature in subclass, needs virtual in C++). Base destructors should be virtual.
- **Construction order:** base constructor, then derived constructor; destruction is the reverse. Rule of three: if you need a destructor, you probably need a copy constructor and copy assignment.
- **Java:** `==` compares references, `equals()` compares content; string literals are interned in the pool; finally always runs (except `System.exit()`); checked exceptions must be handled/declared; an interface can have default methods; a class can implement many interfaces but extend one class.

Part A: MCQs (attempt first, answers follow Part B)

1.1 `int x = 5; int s = sizeof(x++);` What is the value of x afterwards?

- A) 5
- B) 6
- C) 4
- D) Undefined

1.2 `unsigned char c = 255; c++;` What is c?

- A) 256
- B) 0
- C) -1
- D) Undefined

1.3 `int a[5] = {1,2,3}; printf("%d", a[4]);` prints:

- A) A garbage value
- B) 0
- C) 3
- D) Compile error

1.4 What does `printf("%.1f", 7/2*2.0);` print?

- A) 7.0
- B) 6.0
- C) 3.5
- D) 7.5

1.5 C++: class Base has virtual void f() printing "B"; class Derived overrides f() printing "D". Base *p = new Derived; p->f(); prints:

- A) B
- B) D
- C) BD
- D) Compile error

1.6 When a Derived object (Derived extends Base) is created and then destroyed, the order of calls is:

- A) D ctor, B ctor, B dtor, D dtor
- B) B ctor, D ctor, D dtor, B dtor
- C) B ctor, D ctor, B dtor, D dtor
- D) D ctor, B ctor, D dtor, B dtor

1.7 Java: `System.out.println(10 + 20 + "Java" + 10 + 20);` prints:

- A) 30Java30
- B) 30Java1020
- C) 1020Java1020
- D) 1020Java30

1.8 Java: `int i = 'a' + 1; System.out.println(i);` prints:

- A) b
- B) 98
- C) 97
- D) Compile error

1.9 Java: try { return 1; } finally { System.out.println("F"); } What happens?

- A) Returns 1, F is not printed
- B) Prints F, then returns 1
- C) Compile error
- D) Prints F, returns nothing

1.10 C: void f(){ static int c = 0; c++; printf("%d ", c); } Called three times, it prints:

- A) 1 1 1
- B) 1 2 3
- C) 0 1 2
- D) 3 3 3

Part B: Written questions (about 10 marks each)

W1.1 Differentiate function overloading, overriding and hiding. How is run-time polymorphism implemented?

W1.2 Compare an abstract class and an interface in Java. When do you choose each?

W1.3 Find the bugs: (a) char *f(){ char s[10] = "hi"; return s; } (b) char *t = "hi"; t[0] = 'H';

W1.4 Write a swap function in C using pointers and in C++ using references. Why does void swap(int a, int b) not work?

W1.5 Explain shallow copy vs deep copy and the rule of three, with a C++ class that owns a heap array.

Topic 1: Answers and Explanations

MCQ answer key

- 1.1 Answer A.** sizeof is resolved at compile time and does not evaluate its operand, so x++ never runs. (s is 4 on a typical system.)
- 1.2 Answer B.** Unsigned arithmetic wraps modulo $2^8 = 256$, so $255 + 1$ becomes 0. (Signed overflow, by contrast, is undefined.)
- 1.3 Answer B.** When an array is partially initialised, the remaining elements are zero-initialised.
- 1.4 Answer B.** $7/2$ is integer division = 3, then $3 * 2.0 = 6.0$. Evaluation is left to right.
- 1.5 Answer B.** Virtual dispatch picks the function using the run-time type of the object (Derived). Without the virtual keyword the output would be B (static binding).
- 1.6 Answer B.** Construction goes base-first, destruction is the mirror image (derived first).
- 1.7 Answer B.** Evaluation is left to right: $10 + 20 = 30$ (numeric addition), then string concatenation takes over: "30Java" + 10 + 20 = "30Java1020".
- 1.8 Answer B.** 'a' is 97; char is promoted to int, so the sum is the int 98. To print the character b you would cast: (char)('a' + 1).
- 1.9 Answer B.** The finally block always executes before the method actually returns (unless the JVM exits). The return value 1 is already computed.
- 1.10 Answer B.** A static local is initialised once and keeps its value across calls.

Model answers (written)

W1.1 Differentiate function overloading, overriding and hiding. How is run-time polymorphism implemented?

- **Overloading:** same name, different parameter lists, same scope; resolved at compile time.
- **Overriding:** subclass redefines a virtual method with the same signature; resolved at run time by the object's actual type.
- **Hiding:** a derived-class function with the same name (but not virtual/matching) hides all base overloads of that name in C++.
- **Implementation:** each class with virtual functions has a vtable (array of function pointers); each object holds a hidden vptr to its class's vtable. A virtual call loads the vptr, indexes the table, and jumps: one extra indirection.
- **Java:** all instance methods are virtual unless private, static or final.

W1.2 Compare an abstract class and an interface in Java. When do you choose each?

Feature	Abstract class	Interface
State (fields)	Instance fields allowed	Only constants (static final)
Methods	Abstract + concrete, constructors	Abstract, default, static (no constructors)
Inheritance	Single (extends)	Multiple (implements)
Use when	Classes share code and state ("is-a" with common base)	Unrelated classes share a capability (Comparable, Runnable)

Choose an interface to define a contract/capability; choose an abstract class when subclasses need common state or partial implementation.

W1.3 Find the bugs: (a) `char *f(){ char s[10] = "hi"; return s; }` (b) `char *t = "hi"; t[0] = 'H';`

- **(a)** s is a local array on the stack; it is destroyed when f returns, so the returned pointer dangles (UB). Fix: return a malloc-ed buffer (caller frees), use a static array, or pass the buffer in as a parameter.
- **(b)** "hi" is a string literal in read-only memory; modifying it is UB (usually a segmentation fault). Fix: `char t[] = "hi";` (a modifiable copy) or `const char *t`.

W1.4 Write a swap function in C using pointers and in C++ using references. Why does `void swap(int a, int b)` not work?

```
void swap(int *a, int *b){ int t = *a; *a = *b; *b = t; } // call: swap(&x, &y);
void swap(int &a, int &b){ int t = a; a = b; b = t; } // call: swap(x, y);
```

C passes arguments by value: `swap(int a, int b)` exchanges local copies only, and the caller's variables are unchanged. Pointers (or C++ references) give the function access to the original objects.

W1.5 Explain shallow copy vs deep copy and the rule of three, with a C++ class that owns a heap array.

A shallow copy copies the pointer value, so two objects share one buffer; when both destructors run the buffer is freed twice (double free) and edits leak between objects. A deep copy allocates a new buffer and copies the contents.

```
class Buf {
    int n; int *d;
public:
    Buf(int n) : n(n), d(new int[n]) {}
    Buf(const Buf &o) : n(o.n), d(new int[o.n]) { std::copy(o.d, o.d + n, d); }
    Buf& operator=(const Buf &o) {
        if (this != &o) { int *t = new int[o.n]; std::copy(o.d, o.d + o.n, t);
            delete[] d; d = t; n = o.n; }
        return *this; }
    ~Buf() { delete[] d; }
};
```

Rule of three: destructor, copy constructor and copy assignment operator go together. (C++11 adds move operations: rule of five.)

Topic 2: Discrete and Concrete Mathematics, Probability and Statistics

Key facts to revise

- **Counting:** $P(n,r) = n!/(n-r)!$; $C(n,r) = n!/(r!(n-r)!)$; circular arrangements $(n-1)!$; repeated letters $n!/(a!b!...)$; non-negative solutions of $x_1 + \dots + x_k = n$: $C(n+k-1, k-1)$. Inclusion-exclusion: $|A \cup B| = |A| + |B| - |A \cap B|$.
- **Derangements:** $D_1=0, D_2=1, D_3=2, D_4=9, D_5=44, D(n) = (n-1)(D(n-1) + D(n-2))$.
- **Relations/functions:** relations on an n -set = 2^{n^2} ; functions $A \rightarrow B = |B|^{|A|}$; bijections $n!$. Equivalence relation = reflexive + symmetric + transitive.
- **Graphs:** handshaking $\sum(\deg) = 2E$; K_n has $n(n-1)/2$ edges; tree has $n-1$ edges; Euler circuit iff connected and all degrees even; Euler path iff 0 or 2 odd degrees; planar simple graph $E \leq 3V - 6$; number of labelled trees on n vertices n^{n-2} .
- **Recurrences:** $a_n = c_1 a_{n-1} + c_2 a_{n-2}$: solve $r^2 = c_1 r + c_2$; distinct roots r_1, r_2 give $a_n = A r_1^n + B r_2^n$ (repeated root r : $(A + Bn) r^n$).
- **Number theory:** gcd by Euclid; Fermat $a^{p-1} \equiv 1 \pmod p$; modular inverse exists iff $\gcd(a, m) = 1$; $a \pmod m$ cycles: reduce the exponent using the cycle length.
- **Probability:** $P(A|B) = P(A \cap B)/P(B)$; Bayes $P(A|B) = P(B|A)P(A)/P(B)$; $E[X+Y] = E[X] + E[Y]$ always; Binomial mean np , variance npq ; Geometric mean $1/p$; Poisson mean = variance = λ .
- **Statistics:** mean, median, mode; variance = average of squared deviations (sample divides by $n-1$); standard deviation = $\sqrt{\text{variance}}$; normal distribution: about 68/95/99.7% within 1/2/3 sd.

Part A: MCQs (attempt first, answers follow Part B)

2.1 In how many ways can 5 people sit around a circular table?

- | | |
|--------|-------|
| A) 120 | B) 24 |
| C) 60 | D) 12 |

2.2 Number of non-negative integer solutions of $x + y + z = 10$:

- | | |
|--------|-------|
| A) 66 | B) 55 |
| C) 120 | D) 36 |

2.3 Number of derangements of 4 objects:

- | | |
|-------|-------|
| A) 6 | B) 9 |
| C) 12 | D) 24 |

2.4 Number of binary relations on a set with 3 elements:

- | | |
|--------|-------|
| A) 9 | B) 27 |
| C) 512 | D) 64 |

2.5 Number of edges in the complete graph K_6 :

- | | |
|-------|-------|
| A) 12 | B) 15 |
| C) 30 | D) 36 |

2.6 Which complete graph has an Euler circuit?

- | | |
|----------|----------|
| A) K_4 | B) K_5 |
| C) K_6 | D) K_8 |

2.7 Two fair dice are rolled. $P(\text{sum} = 9)$ is:

- | | |
|-----------|-----------|
| A) $1/6$ | B) $1/9$ |
| C) $1/12$ | D) $5/36$ |

2.8 Expected value of the sum of two fair dice:

- | | |
|--------|--------|
| A) 6 | B) 7 |
| C) 7.5 | D) 3.5 |

2.9 $X \sim \text{Binomial}(n = 10, p = 0.5)$. $P(X = 3)$ is about:

- | | |
|----------|----------|
| A) 0.117 | B) 0.205 |
| C) 0.044 | D) 0.300 |

2.10 What is $7^{100} \bmod 5$?

- A) 0
- C) 2

- B) 1
- D) 4

Part B: Written questions (about 10 marks each)

W2.1 Prove by induction that $1 + 3 + 5 + \dots + (2n - 1) = n^2$.

W2.2 Solve $a_n = 5a_{(n-1)} - 6a_{(n-2)}$ with $a_0 = 1$, $a_1 = 4$.

W2.3 Machines A, B, C make 50%, 30%, 20% of items with defect rates 1%, 2%, 3%. An item is defective. Find $P(\text{it came from C})$.

W2.4 Show that among any 6 people, there are 3 mutual friends or 3 mutual strangers.

W2.5 Data: 4, 8, 6, 5, 3, 7. Find mean, median, sample variance and standard deviation.

Topic 2: Answers and Explanations

MCQ answer key

- 2.1 Answer B.** $(5-1)! = 24$. Rotations of the same arrangement are identical.
- 2.2 Answer A.** Stars and bars: $C(10 + 3 - 1, 3 - 1) = C(12, 2) = 66$.
- 2.3 Answer B.** $D_4 = 4!(1 - 1 + 1/2 - 1/6 + 1/24) = 24 - 24 + 12 - 4 + 1 = 9$.
- 2.4 Answer C.** A relation is any subset of $A \times A$, which has 9 pairs: $2^9 = 512$.
- 2.5 Answer B.** $n(n-1)/2 = 6 \times 5 / 2 = 15$.
- 2.6 Answer B.** An Euler circuit needs every degree even. In K_n every degree is $n-1$, which is even only for odd n . So K_5 (degree 4).
- 2.7 Answer B.** Favourable: $(3,6), (4,5), (5,4), (6,3) = 4$ of $36 = 1/9$.
- 2.8 Answer B.** By linearity, $E = 3.5 + 3.5 = 7$.
- 2.9 Answer A.** $C(10,3)/2^{10} = 120/1024 = 0.117$.
- 2.10 Answer B.** $7 \equiv 2 \pmod{5}$. Powers of 2 mod 5 cycle 2,4,3,1 (period 4). 100 is a multiple of 4, so the result is 1. (Or Fermat: $7^4 \equiv 1 \pmod{5}$.)

Model answers (written)

W2.1 Prove by induction that $1 + 3 + 5 + \dots + (2n - 1) = n^2$.

- **Base ($n = 1$):** left side = 1, right side = $1^2 = 1$. True.
- **Hypothesis:** assume $1 + 3 + \dots + (2k - 1) = k^2$.
- **Step:** add the next odd number $(2k + 1)$: $k^2 + 2k + 1 = (k + 1)^2$. This is the formula for $n = k + 1$.
- By induction the statement holds for all $n \geq 1$.

W2.2 Solve $a_n = 5a_{n-1} - 6a_{n-2}$ with $a_0 = 1, a_1 = 4$.

- Characteristic equation: $r^2 - 5r + 6 = 0$, so $(r - 2)(r - 3) = 0$, $r = 2$ or 3 .
- General solution: $a_n = A \cdot 2^n + B \cdot 3^n$.
- a_0 : $A + B = 1$. a_1 : $2A + 3B = 4$. Subtract $2 \times$ first from second: $B = 2$, so $A = -1$.
- $a_n = 2 \cdot 3^n - 2^n$. Check: $a_2 = 5 \cdot 4 - 6 \cdot 1 = 14$ and $2 \cdot 9 - 4 = 14$. Correct.

W2.3 Machines A, B, C make 50%, 30%, 20% of items with defect rates 1%, 2%, 3%. An item is defective. Find P(it came from C).

- $P(\text{defect}) = 0.5(0.01) + 0.3(0.02) + 0.2(0.03) = 0.005 + 0.006 + 0.006 = 0.017$ (total probability).
- Bayes: $P(C | \text{defect}) = 0.006 / 0.017 = \mathbf{0.353}$ (about 35.3%).
- Note that C makes only 20% of items but accounts for 35% of defects.

W2.4 Show that among any 6 people, there are 3 mutual friends or 3 mutual strangers.

- Pick person P. Among the other 5, P is friends with at least 3 or a stranger to at least 3 (pigeonhole: 5 people, 2 categories).
- Suppose P is friends with X, Y, Z (the other case is symmetric).
- If any two of X, Y, Z are friends, they form a friend triangle with P. Otherwise X, Y, Z are pairwise strangers: a stranger triangle.
- Either way the claim holds. This is the Ramsey number $R(3,3) = 6$; five people are not enough (a pentagon colouring is a counter-example).

W2.5 Data: 4, 8, 6, 5, 3, 7. Find mean, median, sample variance and standard deviation.

- Mean = $33/6 = \mathbf{5.5}$. Sorted: 3, 4, 5, 6, 7, 8; median = $(5 + 6)/2 = \mathbf{5.5}$.
- Deviations from the mean: -1.5, 2.5, 0.5, -0.5, -2.5, 1.5; squares: 2.25, 6.25, 0.25, 0.25, 6.25, 2.25; sum = 17.5.
- Sample variance = $17.5/5 = \mathbf{3.5}$; sample sd = $\sqrt{3.5} = \mathbf{1.87}$. (Population variance = $17.5/6 = 2.92$.)

Topic 3: Data Structures and Algorithms

Key facts to revise

- **Sorting:** bubble/insertion/selection $O(n^2)$ (insertion best case $O(n)$); merge sort $O(n \log n)$, stable, $O(n)$ extra space; quick sort average $O(n \log n)$, worst $O(n^2)$; heap sort $O(n \log n)$ in place, not stable; counting sort $O(n + k)$.
- **Trees:** BST ops $O(h)$; AVL keeps $|\text{balance}| \leq 1$ (rotations LL, RR, LR, RL); heap: build $O(n)$, insert/delete $O(\log n)$; complete binary tree of n nodes has height $\text{floor}(\log_2 n)$. Catalan number C_n counts binary tree shapes ($C_3 = 5$).
- **Graphs:** BFS/DFS $O(V + E)$ with lists, $O(V^2)$ with a matrix; Dijkstra $O((V+E) \log V)$ (no negative edges); Bellman-Ford $O(VE)$ (detects negative cycles); Floyd-Warshall $O(V^3)$; Kruskal $O(E \log E)$; Prim $O(E \log V)$; topological sort $O(V + E)$ for DAGs.
- **Master theorem** $T(n) = aT(n/b) + f(n)$: compare $f(n)$ with $n^{\log_b a}$. Smaller: $\Theta(n^{\log_b a})$. Equal: $\Theta(n^{\log_b a} \log n)$. Larger (with regularity): $\Theta(f(n))$.
- **Hashing:** load factor = n/m ; chaining vs open addressing (linear, quadratic, double hashing); average lookup $O(1)$.
- **Dynamic programming:** optimal substructure + overlapping subproblems (LCS, 0/1 knapsack, LIS, matrix-chain, edit distance). Greedy: activity selection, Huffman, fractional knapsack, Kruskal/Prim.
- **Linked lists:** $O(1)$ insert/delete at a known node (given predecessor for singly); head+tail pointers give $O(1)$ queue operations; Floyd cycle detection with slow/fast pointers.

Part A: MCQs (attempt first, answers follow Part B)

3.1 The height (in edges) of a complete binary tree with n nodes is:

- | | |
|----------|-----------------------------|
| A) n | B) $\text{floor}(\log_2 n)$ |
| C) $n/2$ | D) $\sqrt{n}$ |

3.2 Number of structurally different binary trees with 3 nodes:

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 6 |

3.3 Inorder: 4 2 5 1 3. Preorder: 1 2 4 5 3. The postorder is:

- | | |
|--------------|--------------|
| A) 4 5 2 3 1 | B) 4 2 5 3 1 |
| C) 5 4 2 3 1 | D) 4 5 3 2 1 |

3.4 10, 20, 30 are inserted in this order into an empty AVL tree. The root after rebalancing is:

- | | |
|-------|----------------------------|
| A) 10 | B) 20 |
| C) 30 | D) 10 (no rotation needed) |

3.5 Bottom-up build-heap (max-heap) on [3, 9, 2, 1, 4, 5] gives:

- | | |
|------------------|------------------|
| A) [9,4,5,1,3,2] | B) [9,5,4,3,1,2] |
| C) [9,3,5,1,4,2] | D) [9,4,5,3,1,2] |

3.6 BFS on a graph stored as an adjacency matrix runs in:

- | | |
|------------------|-------------|
| A) $O(V + E)$ | B) $O(V^2)$ |
| C) $O(E \log V)$ | D) $O(VE)$ |

3.7 $T(n) = 4T(n/2) + n$ has solution:

- | | |
|------------------|-------------------------|
| A) $\Theta(n)$ | B) $\Theta(n \log n)$ |
| C) $\Theta(n^2)$ | D) $\Theta(n^2 \log n)$ |

3.8 Hash table of size 7, $h(k) = k \bmod 7$, linear probing. Insert 15, 22, 29, 8 in order. The index where 8 lands:

- | | |
|------|------|
| A) 1 | B) 2 |
| C) 3 | D) 4 |

3.9 Number of spanning trees of K_5 :

- | | |
|--------|--------|
| A) 25 | B) 125 |
| C) 120 | D) 16 |

3.10 Value of the postfix expression 5 3 + 2 *:

A) 11

B) 16

C) 13

D) 10

Part B: Written questions (about 10 marks each)

W3.1 Compare Kruskal's and Prim's algorithms for minimum spanning trees. Give complexities and when to prefer each.

W3.2 Write a function to reverse a singly linked list iteratively. State its complexity.

W3.3 Find the LCS of $X = \text{ABCB DAB}$ and $Y = \text{BDCABA}$. Give the recurrence and the complexity.

W3.4 Explain merge sort: algorithm, recurrence, stability and space.

W3.5 Explain the 0/1 knapsack DP. Why is it called pseudo-polynomial and why does greedy fail?

W3.6 Explain topological sorting (Kahn's algorithm) and how to detect a cycle with it.

Topic 3: Answers and Explanations

MCQ answer key

- 3.1 Answer B.** Levels fill left to right, so the number of nodes doubles per level: height = floor(log₂ n).
- 3.2 Answer C.** Catalan number $C_3 = (1/4) C(6,3) = 5$.
- 3.3 Answer A.** Preorder gives root 1. Inorder splits into left {4,2,5} and right {3}. Left subtree root is 2 with children 4 and 5. Postorder: 4 5 2, then 3, then 1.
- 3.4 Answer B.** Inserting 30 makes node 10 unbalanced (right-right case). A left rotation at 10 makes 20 the root with children 10 and 30.
- 3.5 Answer A.** Sift down from index 2: swap 2 with 5 -> [3,9,5,1,4,2]. Index 1 (9) is fine. Index 0: swap 3 with 9 -> [9,3,5,1,4,2]; then 3 sifts below and swaps with 4 -> [9,4,5,1,3,2].
- 3.6 Answer B.** Finding the neighbours of each vertex scans a whole row of V entries, for V vertices.
- 3.7 Answer C.** $n^{\log_2 4} = n^2$ grows faster than $f(n) = n$, so case 1: Theta(n^2).
- 3.8 Answer D.** 15 -> 1. 22 -> 1 is taken, so 2. 29 -> 1, 2 taken, so 3. 8 -> 1, 2, 3 taken, so 4.
- 3.9 Answer B.** Cayley's formula: $n^{(n-2)} = 5^3 = 125$.
- 3.10 Answer B.** Push 5, 3; + gives 8; push 2; * gives 16, i.e. $(5 + 3) * 2$.

Model answers (written)

W3.1 Compare Kruskal's and Prim's algorithms for minimum spanning trees. Give complexities and when to prefer each.

- **Kruskal:** sort edges by weight; add the next lightest edge if it joins two different components (checked with union-find). $O(E \log E)$. Works on a forest of components; good for sparse graphs and when edges are already sorted.
- **Prim:** grow one tree from a start vertex, always adding the cheapest edge leaving the tree (priority queue). $O(E \log V)$ with a binary heap, $O(E + V \log V)$ with a Fibonacci heap. Good for dense graphs.
- Both rely on the cut property and both give an MST with $V - 1$ edges (unique if all weights are distinct).

W3.2 Write a function to reverse a singly linked list iteratively. State its complexity.

```
Node* reverse(Node *head){
    Node *prev = NULL, *cur = head;
    while (cur != NULL){
        Node *next = cur->next; // remember the rest
        cur->next = prev;       // flip the link
        prev = cur;             // advance
        cur = next;
    }
    return prev;                // new head
}
```

Time $O(n)$, extra space $O(1)$. Recursive version: reverse the rest, then make $head \rightarrow next \rightarrow next = head$ and $head \rightarrow next = NULL$ ($O(n)$ stack).

W3.3 Find the LCS of $X = ABCBDAB$ and $Y = BDCABA$. Give the recurrence and the complexity.

- Let $c[i][j]$ be the LCS length of the first i letters of X and first j letters of Y . $c[i][0] = c[0][j] = 0$.
- If $X[i] = Y[j]$: $c[i][j] = c[i-1][j-1] + 1$. Otherwise $c[i][j] = \max(c[i-1][j], c[i][j-1])$.
- Filling the table takes $O(mn)$ time and space ($O(\min(m,n))$ space if only the length is needed). Trace back from $c[m][n]$ to recover a subsequence.
- For these strings the LCS length is **4**, for example **BCBA**.

W3.4 Explain merge sort: algorithm, recurrence, stability and space.

- Divide the array in two halves, sort each recursively, then merge the two sorted halves with two pointers.
- Recurrence $T(n) = 2T(n/2) + \Theta(n)$, so **Theta($n \log n$)** in best, average and worst case (master theorem case 2).
- **Stable** if the merge takes from the left half on ties. Needs **$O(n)$ extra space** for arrays (linked lists can be merged in $O(1)$ extra). Good for external sorting and linked lists; quick sort is usually faster in memory because of cache behaviour.

W3.5 Explain the 0/1 knapsack DP. Why is it called pseudo-polynomial and why does greedy fail?

- Let $K[i][w]$ be the best value using the first i items with capacity w . $K[i][w] = K[i-1][w]$ if $wt_i > w$, else $\max(K[i-1][w], K[i-1][w - wt_i] + val_i)$. Answer $K[n][W]$.
- Time and space $O(nW)$. W is the numeric value of the capacity, not the number of bits needed to write it, so the running time is exponential in the input length: pseudo-polynomial.
- Greedy by value/weight fails for 0/1 items: capacity 50, items $(wt, val) = (10,60), (20,100), (30,120)$: greedy takes the first two (160) but the best is items 2 and 3 (220). Greedy is optimal only for the fractional version.

W3.6 Explain topological sorting (Kahn's algorithm) and how to detect a cycle with it.

- Compute the in-degree of every vertex. Put all vertices with in-degree 0 in a queue.
- Repeatedly remove a vertex from the queue, append it to the order, and decrement the in-degree of its neighbours; enqueue any that reach 0.
- If the final order has fewer than V vertices, the graph has a cycle (no topological order exists). Time $O(V + E)$.
- Alternative: DFS and output vertices in reverse finishing order.

Topic 4: Compiler Design and Theory of Computation

Key facts to revise

- **Hierarchy:** Regular (DFA/NFA/regex) in CFL (PDA) in context-sensitive (LBA) in decidable (halting TM) in recursively enumerable (TM). Regular languages are closed under union, intersection, complement, concatenation, star; CFLs are closed under union, concatenation, star but NOT intersection or complement.
- **Pumping lemma (regular):** for w in L with $|w| \geq p$, $w = xyz$, $|xy| \leq p$, $|y| \geq 1$, and $xy^i z$ in L for all $i \geq 0$. Used only to prove non-regularity. Typical non-regular: $a^n b^n$, ww^R over unbounded n , $a^{(n^2)}$, primes. $a^n b^n c^n$ is not context-free.
- **DFA facts:** minimal DFA for strings ending in a fixed k -length pattern needs $k + 1$ states; divisible-by- k needs k states; subset construction can blow up to 2^n states.
- **Decidability:** halting problem, emptiness of a TM language, equivalence of CFGs, and any non-trivial property of TM languages (Rice) are undecidable. Halting is recursively enumerable but not decidable.
- **P/NP:** P in NP; NP-complete = NP $\cap$ NP-hard; SAT (Cook-Levin), 3-SAT, clique, vertex cover, Hamiltonian cycle, subset sum, 3-colouring are NP-complete; 2-SAT and shortest path are in P.
- **Compiler phases:** lexical analysis (tokens), syntax analysis (parse tree), semantic analysis (types, scopes), intermediate code (three-address code), optimisation, code generation. Symbol table and error handling serve all phases.
- **Parsing:** LL(1) needs no left recursion, no common prefixes and disjoint FIRST/FOLLOW sets. Power: $LR(0) < SLR(1) < LALR(1) < CLR(1)$. SLR and LALR have the same number of states as $LR(0)$; $CLR(1)$ has the most.
- **Optimisation:** constant folding, constant propagation, dead-code elimination, common subexpression elimination, loop-invariant code motion, strength reduction ($x*2 \rightarrow x+x$).

Part A: MCQs (attempt first, answers follow Part B)

4.1 Which of these languages is regular?

- | | |
|-----------------------------------|--------------------------------------|
| A) $\{a^n b^n \mid n \geq 0\}$ | B) $\{w w^R \mid w \in \{a,b\}^*\}$ |
| C) $\{a^n b^m \mid n, m \geq 0\}$ | D) $\{a^p \mid p \text{ is prime}\}$ |

4.2 Minimum number of states in a DFA over $\{0,1\}$ accepting strings that end in 01:

- | | |
|------|------|
| A) 2 | B) 3 |
| C) 4 | D) 5 |

4.3 Context-free languages are NOT closed under:

- | | |
|----------------|------------------|
| A) Union | B) Concatenation |
| C) Kleene star | D) Intersection |

4.4 The halting problem is:

- | | |
|------------------------------------|---|
| A) Decidable | B) Undecidable but recursively enumerable |
| C) Not even recursively enumerable | D) Regular |

4.5 Which statement about LR parser state counts is true?

- | | |
|---------------------------------------|---|
| A) SLR has more states than LALR | B) $CLR(1)$ has the most states; SLR and LALR have equal counts |
| C) LALR has more states than $CLR(1)$ | D) All have the same number |

4.6 The grammar $E \rightarrow E + E \mid id$ is:

- | | |
|------------|------------------|
| A) LL(1) | B) Ambiguous |
| C) Regular | D) Left factored |

4.7 A type mismatch (adding an int to a string) is detected in:

- | | |
|----------------------|--------------------|
| A) Lexical analysis | B) Syntax analysis |
| C) Semantic analysis | D) Code generation |

4.8 Replacing $2 * 3.14$ by 6.28 at compile time is called:

- | | |
|--------------------------|-------------------------------------|
| A) Dead-code elimination | B) Constant folding |
| C) Strength reduction | D) Common subexpression elimination |

4.9 Number of states in the minimal DFA for $(a|b)^*abb$:

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 8 |

4.10 Which of these problems is NP-complete?

- | | |
|------------------|--------------------------|
| A) Shortest path | B) 2-SAT |
| C) 3-SAT | D) Minimum spanning tree |

Part B: Written questions (about 10 marks each)

W4.1 Prove that $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

W4.2 Compute FIRST, FOLLOW and the LL(1) table for: $E \rightarrow TE'$, $E' \rightarrow +TE' \mid \text{eps}$, $T \rightarrow FT'$, $T' \rightarrow *FT' \mid \text{eps}$, $F \rightarrow (E) \mid \text{id}$.

W4.3 Explain the phases of a compiler using the statement $\text{position} = \text{initial} + \text{rate} * 60$.

W4.4 Design a DFA that accepts binary strings representing numbers divisible by 3.

W4.5 Distinguish decidable, recursively enumerable and undecidable languages. Sketch why the halting problem is undecidable.

Topic 4: Answers and Explanations

MCQ answer key

- 4.1 Answer C.** a^*b^* needs no counting: any number of a's followed by any number of b's. The others need unbounded memory (counting or matching).
- 4.2 Answer B.** States track: no progress, last symbol 0, last two symbols 01 (accepting). That is 3 states.
- 4.3 Answer D.** Example: $\{a^n b^n c^m\}$ and $\{a^m b^n c^n\}$ are CFLs but their intersection $\{a^n b^n c^n\}$ is not.
- 4.4 Answer B.** A universal TM can simulate a program and accept if it halts (semi-decidable), but no TM can decide it for all inputs.
- 4.5 Answer B.** LALR merges CLR(1) states with identical cores, giving the same number of states as SLR/LR(0).
- 4.6 Answer B.** The string $id + id + id$ has two different parse trees (left- vs right-associative). It is also left-recursive, so not LL(1).
- 4.7 Answer C.** Type checking is part of semantic analysis. The parser only checks grammatical structure.
- 4.8 Answer B.** Expressions whose operands are constants are evaluated by the compiler.
- 4.9 Answer B.** Classic textbook example: four states (nothing matched, a, ab, abb).
- 4.10 Answer C.** 3-SAT is NP-complete. The others have polynomial-time algorithms.

Model answers (written)

W4.1 Prove that $L = \{a^n b^n \mid n \geq 0\}$ is not regular.

- Assume L is regular with pumping length p . Take $w = a^p b^p$, so $|w| \geq p$.
- Any split $w = xyz$ with $|xy| \leq p$ and $|y| \geq 1$ forces y to consist only of a's: $y = a^k$ with $k \geq 1$.
- Pump up ($i = 2$): $xy^2z = a^{p+k} b^p$ has more a's than b's, so it is not in L .
- This contradicts the pumping lemma, so L is not regular.

(Remember: the lemma can only prove non-regularity; it is necessary, not sufficient.)

W4.2 Compute FIRST, FOLLOW and the LL(1) table for: $E \rightarrow TE'$, $E' \rightarrow +TE'$ | ϵ , $T \rightarrow FT'$, $T' \rightarrow *FT'$ | ϵ , $F \rightarrow (E)$ | id .

- $FIRST(E) = FIRST(T) = FIRST(F) = \{ (, id \}$. $FIRST(E') = \{ +, \epsilon \}$. $FIRST(T') = \{ *, \epsilon \}$.
- $FOLLOW(E) = FOLLOW(E') = \{), \$ \}$. $FOLLOW(T) = FOLLOW(T') = \{ +,), \$ \}$. $FOLLOW(F) = \{ *, +,), \$ \}$.
- Table entries: $M[E, id] = M[E, (] = E \rightarrow TE'$. $M[E', +] = E' \rightarrow +TE'$; $M[E',)]$ = $M[E', \$]$ = $E' \rightarrow \epsilon$. $M[T, id] = M[T, (] = T \rightarrow FT'$. $M[T', *] = T' \rightarrow *FT'$; $M[T', +] = M[T',)]$ = $M[T', \$]$ = $T' \rightarrow \epsilon$. $M[F, id] = F \rightarrow id$; $M[F, (] = F \rightarrow (E)$.
- No cell has two entries, so the grammar is LL(1). Rule for ϵ -productions: use FOLLOW(A).

W4.3 Explain the phases of a compiler using the statement $position = initial + rate * 60$.

- Lexical analysis:** tokens $id(position)$, $=$, $id(initial)$, $+$, $id(rate)$, $*$, $num(60)$.
- Syntax analysis:** parse tree respecting precedence: $initial + (rate * 60)$.
- Semantic analysis:** type checking; if $rate$ is float, insert an int-to-float conversion for 60.
- Intermediate code:** $t1 = 60.0$; $t2 = rate * t1$; $t3 = initial + t2$; $position = t3$.
- Optimisation:** fold the conversion of 60 at compile time, remove the extra temporaries: $t1 = rate * 60.0$; $position = initial + t1$.
- Code generation:** map to registers/instructions (load, multiply, add, store). The symbol table stores names and types throughout.

W4.4 Design a DFA that accepts binary strings representing numbers divisible by 3.

- States are the remainder so far: r_0 , r_1 , r_2 . Reading bit b turns the number v into $2v + b$, so the new remainder is $(2r + b) \bmod 3$.
- Transitions: $r_0 \xrightarrow{0} r_0$, $r_0 \xrightarrow{1} r_1$; $r_1 \xrightarrow{0} r_2$, $r_1 \xrightarrow{1} r_0$; $r_2 \xrightarrow{0} r_1$, $r_2 \xrightarrow{1} r_2$.
- Start state r_0 , accepting state r_0 (so the empty string and 0 are accepted). Test: 110 (6) goes r_0, r_1, r_0, r_0 : accepted. 101 (5): r_0, r_1, r_2, r_2 : rejected.

W4.5 Distinguish decidable, recursively enumerable and undecidable languages. Sketch why the halting problem is undecidable.

- **Decidable:** some TM always halts, accepting exactly the members. **Recursively enumerable:** a TM accepts members (may loop forever on non-members). **Undecidable:** no TM decides it.
- **Proof sketch (diagonalisation):** suppose $H(P, x)$ decides whether program P halts on input x . Build $D(P)$: if $H(P, P)$ says "halts" then loop forever, else halt. Run D on itself: $D(D)$ halts iff H says $D(D)$ does not halt: contradiction. So H cannot exist.
- Reductions spread undecidability: if halting reduces to problem X , X is undecidable. Rice's theorem: every non-trivial property of a TM's language is undecidable.

Topic 5: Database Systems

Key facts to revise

- **Keys and FDs:** superkey, candidate key, primary key. Closure $X^+ =$ all attributes determined by X (Armstrong axioms: reflexivity, augmentation, transitivity). A candidate key has closure = all attributes; attributes never on any right-hand side must be in every key.
- **Normal forms:** 1NF atomic values; 2NF no partial dependency on part of a key; 3NF for every FD $X \rightarrow Y$, X is a superkey or Y is a prime attribute; BCNF X must always be a superkey. BCNF is stricter than 3NF; decomposition into 3NF is always lossless and dependency-preserving, BCNF may lose dependencies.
- **SQL:** JOIN types (INNER, LEFT, RIGHT, FULL); WHERE filters rows before grouping, HAVING filters groups after; COUNT(*) counts rows, COUNT(col) skips NULLs; comma without WHERE is a Cartesian product; DELETE (DML, row by row, rollback) vs TRUNCATE (DDL, all rows) vs DROP (removes the table).
- **Transactions (ACID):** Atomicity, Consistency, Isolation, Durability. Anomalies: dirty read, non-repeatable read, phantom read, lost update. Isolation levels: read uncommitted, read committed, repeatable read, serializable.
- **Serializability:** build a precedence graph (edge $T_i \rightarrow T_j$ for conflicting operations where T_i goes first on the same item, at least one a write). Acyclic means conflict serializable. Two-phase locking (growing then shrinking) guarantees serializability; strict 2PL holds write locks until commit. Deadlocks: detect with a wait-for graph, or prevent (wait-die, wound-wait).
- **Indexing:** B+ tree (all data pointers in linked leaves, good for range queries, order n means at most n pointers per node and n-1 keys); hash index (equality only); clustered index orders the data file (one per table); dense vs sparse index.
- **Recovery:** write-ahead logging, undo (uncommitted) and redo (committed) using log records; checkpoints limit recovery work.

Part A: MCQs (attempt first, answers follow Part B)

5.1 R(A,B,C,D,E) with $A \rightarrow B$, $BC \rightarrow D$, $D \rightarrow E$. The closure $(AC)^+$ is:

- | | |
|----------------|--------------|
| A) {A,C} | B) {A,B,C,D} |
| C) {A,B,C,D,E} | D) {A,C,D,E} |

5.2 R has 4 tuples and S has 5 tuples with no common attributes. The natural join $R \Join S$ has:

- | | |
|-------------|--------------|
| A) 9 tuples | B) 20 tuples |
| C) 0 tuples | D) 5 tuples |

5.3 Which command removes all rows but keeps the table definition, and is usually treated as DDL?

- | | |
|-------------|-----------|
| A) DELETE | B) DROP |
| C) TRUNCATE | D) REMOVE |

5.4 T1 reads a row twice and sees different values because T2 committed an update in between. This is a:

- | | |
|-----------------|------------------------|
| A) Dirty read | B) Non-repeatable read |
| C) Phantom read | D) Lost update |

5.5 Schedule: $r1(x)$; $w2(x)$; $w1(x)$. Is it conflict serializable?

- | | |
|----------------------------------|----------------------------------|
| A) Yes, equivalent to T1 then T2 | B) Yes, equivalent to T2 then T1 |
| C) No | D) Cannot be determined |

5.6 $A = \{1,2,3\}$ and $B = \{2,3,4\}$ (join on id). Number of rows in A FULL OUTER JOIN B:

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 6 |

5.7 Which index type best supports range queries such as salary BETWEEN 30000 AND 50000?

- | | |
|-----------------|---------------------------|
| A) Hash index | B) B+ tree index |
| C) No index can | D) Bitmap on a unique key |

5.8 R(A,B,C) with FDs $AB \rightarrow C$ and $C \rightarrow B$. The highest normal form is:

- | | |
|--------------------|---------------------|
| A) BCNF | B) 3NF but not BCNF |
| C) 2NF but not 3NF | D) 1NF only |

5.9 A table has 5 rows; column x holds 2 NULLs. COUNT(x) returns:

- | | |
|-------------|-------------|
| A) 5 | B) 2 |
| C) 3 | D) 0 |

5.10 The HAVING clause is used to:

- | | |
|---------------------------------------|--|
| A) Filter rows before grouping | B) Filter groups after GROUP BY |
| C) Sort the result | D) Join two tables |

Part B: Written questions (about 10 marks each)

W5.1 Write SQL to find the second highest salary from Employee(id, name, salary, dept_id).

W5.2 Departments(dept_id, dept_name), Employee(id, name, salary, dept_id). List each department name with average salary, only for departments with more than 5 employees.

W5.3 R(A,B,C,D,E) has FDs AB → C, C → D, D → E. Find the candidate key, the highest normal form and a 3NF decomposition.

W5.4 Explain two-phase locking. Why does it guarantee serializability, and how do you handle deadlocks?

W5.5 Explain indexing: clustered vs non-clustered, and why DBMSs prefer B+ trees to binary trees or B trees.

Topic 5: Answers and Explanations

MCQ answer key

- 5.1 Answer C.** A->B adds B. BC->D adds D. D->E adds E. So AC determines everything and AC is a key.
- 5.2 Answer B.** With no common attribute the natural join degenerates to the Cartesian product: $4 \times 5 = 20$.
- 5.3 Answer C.** TRUNCATE empties the table quickly and resets identity counters. DROP removes the table itself.
- 5.4 Answer B.** Same row, different value = non-repeatable read. Phantom = new rows appear in a repeated range query. Dirty = reading uncommitted data.
- 5.5 Answer C.** $r1(x)$ before $w2(x)$ gives $T1 \rightarrow T2$; $w2(x)$ before $w1(x)$ gives $T2 \rightarrow T1$. The precedence graph has a cycle.
- 5.6 Answer B.** Matches: 2 and 3. Plus unmatched 1 (from A) and 4 (from B): 4 rows.
- 5.7 Answer B.** B+ tree leaves are sorted and linked, so a range scan walks the leaf chain. Hash indexes only support equality.
- 5.8 Answer B.** Keys are AB and AC, so B is a prime attribute: $C \rightarrow B$ satisfies 3NF. C is not a superkey, so BCNF is violated.
- 5.9 Answer C.** COUNT(column) ignores NULL values; COUNT(*) would return 5.
- 5.10 Answer B.** HAVING applies conditions to aggregate results per group. WHERE filters individual rows before aggregation.

Model answers (written)

W5.1 Write SQL to find the second highest salary from Employee(id, name, salary, dept_id).

```
SELECT MAX(salary) FROM Employee
WHERE salary < (SELECT MAX(salary) FROM Employee);

-- alternative (handles duplicates of the top salary):
SELECT DISTINCT salary FROM Employee
ORDER BY salary DESC LIMIT 1 OFFSET 1;
```

The first query removes the maximum and takes the next maximum. Use DISTINCT/DENSE_RANK() when several employees share the top salary.

W5.2 Departments(dept_id, dept_name), Employee(id, name, salary, dept_id). List each department name with average salary, only for departments with more than 5 employees.

```
SELECT d.dept_name, AVG(e.salary) AS avg_salary
FROM Departments d JOIN Employee e ON e.dept_id = d.dept_id
GROUP BY d.dept_id, d.dept_name
HAVING COUNT(*) > 5;
```

JOIN attaches names, GROUP BY makes one group per department, and HAVING filters groups by the aggregate COUNT(*). Using WHERE COUNT(*) > 5 would be an error.

W5.3 R(A,B,C,D,E) has FDs $AB \rightarrow C$, $C \rightarrow D$, $D \rightarrow E$. Find the candidate key, the highest normal form and a 3NF decomposition.

- A and B never appear on a right-hand side, so they belong to every key. $AB^+ = \{A,B,C,D,E\}$, so **AB is the only candidate key**.
- $C \rightarrow D$ and $D \rightarrow E$ have non-superkey left sides and non-prime right sides: transitive dependencies. The relation is in 2NF (no partial dependency, since the key AB is not split by any FD) but **not 3NF**.
- Decomposition:** $R1(A,B,C)$ key AB; $R2(C,D)$ key C; $R3(D,E)$ key D. It is lossless ($R1$ keeps the key AB) and preserves all FDs; every relation is in BCNF.

W5.4 Explain two-phase locking. Why does it guarantee serializability, and how do you handle deadlocks?

- 2PL:** every transaction has a growing phase (acquire locks, none released) and a shrinking phase (release locks, none acquired). The lock point (end of growing phase) gives an equivalent serial order.
- Strict 2PL** holds exclusive locks until commit/abort, avoiding cascading aborts.
- Deadlock:** $T1$ holds A and waits for B while $T2$ holds B and waits for A. Handle by detection (cycle in the wait-for graph, abort a victim), prevention (wait-die / wound-wait by timestamps), or timeouts.

- ACID example: transfer 100 from account X to Y. Atomicity = both updates or neither; Consistency = total balance unchanged; Isolation = other transactions never see half of the transfer; Durability = once committed it survives a crash (write-ahead log).

W5.5 Explain indexing: clustered vs non-clustered, and why DBMSs prefer B+ trees to binary trees or B trees.

- **Clustered index:** the table rows are physically ordered by the index key (one per table). **Non-clustered:** a separate structure of (key, pointer) entries; many per table.
- **B+ tree:** balanced, high fan-out so the height is tiny (3-4 levels for millions of rows), each node fits a disk page, internal nodes hold only keys (more keys per page than a B tree), and all records are reachable from linked leaves, making range scans and full scans sequential.
- A binary tree is too deep (one disk access per level). A leaf node of a B+ tree of order n holds at most $n - 1$ keys with record pointers.
- Cost model: an index lookup costs about the tree height in I/O operations, versus a full table scan.

Topic 6: Computer Networks and Security

Key facts to revise

- **Layers:** OSI (physical, data link, network, transport, session, presentation, application) vs TCP/IP (link, internet, transport, application). ARP maps IP to MAC; ICMP reports errors (ping); DHCP gives IP configuration (Discover, Offer, Request, Ack); DNS resolves names, queries normally use UDP port 53 (TCP for zone transfers and large answers).
- **Subnetting:** block size = $256 - \text{mask octet}$; usable hosts = $2^{(32 - \text{prefix})} - 2$. /24 = 254, /25 = 126, /26 = 62, /27 = 30, /28 = 14, /29 = 6, /30 = 2. Network address = IP AND mask; broadcast = last address of the block.
- **Capacity:** Nyquist (noiseless) = $2B \log_2(V)$ bps; Shannon = $B \log_2(1 + \text{SNR})$ bps (SNR as a ratio, dB = $10 \log_{10} \text{ratio}$).
- **Error control:** CRC with a generator of degree r appends r zero bits before division; Hamming needs r parity bits with $2^r \geq m + r + 1$; single parity detects odd numbers of bit errors.
- **MAC / ARQ:** pure ALOHA max efficiency 18.4% ($1/2e$), slotted ALOHA 36.8% ($1/e$); CSMA/CD in Ethernet needs a minimum frame time of $2 \times \text{propagation}$. Go-Back-N window $\leq 2^n - 1$, Selective Repeat window $\leq 2^{(n-1)}$ for n -bit sequence numbers. Half duplex = both ways but one at a time.
- **TCP:** three-way handshake SYN, SYN+ACK, ACK; flow control by the receiver window; congestion control with slow start (window doubles per RTT up to ssthresh), congestion avoidance ($+1 \text{ MSS per RTT}$), on loss ssthresh = $\text{cwnd}/2$. UDP is connectionless, no delivery guarantee.
- **Routing:** distance vector (Bellman-Ford, RIP; count-to-infinity problem) vs link state (Dijkstra with full topology, OSPF). BGP is the inter-domain protocol.
- **Security:** CIA (confidentiality, integrity, availability); symmetric (AES) fast, shared secret; asymmetric (RSA) slow, key pairs; hashes (SHA-256) for integrity; digital signature = sign with sender private key, verify with public key; TLS uses asymmetric crypto to agree a symmetric session key. Attacks: SQL injection, XSS, CSRF, MITM, ARP spoofing, DoS/DDoS, SYN flood, DHCP starvation, phishing.

Part A: MCQs (attempt first, answers follow Part B)

6.1 Number of usable host addresses in a /23 network:

- | | |
|--------|---------|
| A) 254 | B) 510 |
| C) 512 | D) 1022 |

6.2 Which protocol resolves an IP address to a MAC address on a LAN?

- | | |
|--------|---------|
| A) DNS | B) DHCP |
| C) ARP | D) ICMP |

6.3 A channel has $B = 3000 \text{ Hz}$ and $\text{SNR} = 1000$ (ratio). Shannon capacity is about:

- | | |
|-------------|------------|
| A) 3 kbps | B) 30 kbps |
| C) 300 kbps | D) 6 kbps |

6.4 In the TCP three-way handshake the second segment carries the flags:

- | | |
|--------|--------------|
| A) SYN | B) SYN + ACK |
| C) ACK | D) FIN |

6.5 Maximum throughput efficiency of pure ALOHA:

- | | |
|----------|----------|
| A) 36.8% | B) 18.4% |
| C) 50% | D) 100% |

6.6 CRC uses generator 1011 (degree 3). How many zero bits are appended to the data before division?

- | | |
|------|-------|
| A) 4 | B) 3 |
| C) 2 | D) 11 |

6.7 Hamming code: how many parity bits are needed for 7 data bits?

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 7 |

6.8 Selective Repeat ARQ with 3-bit sequence numbers: maximum window size is:

- | | |
|------|------|
| A) 7 | B) 8 |
| C) 4 | D) 3 |

6.9 DNS queries normally use:

- | | |
|----------------|----------------|
| A) TCP port 53 | B) UDP port 53 |
| C) UDP port 67 | D) TCP port 80 |

6.10 In a digital signature the sender signs the message hash with:

- | | |
|--------------------------|---------------------------|
| A) Receiver's public key | B) Sender's private key |
| C) Sender's public key | D) Receiver's private key |

Part B: Written questions (about 10 marks each)

W6.1 Divide 192.168.1.0/24 into 4 equal subnets. Give network, usable range and broadcast for each. Then find the network and broadcast of 192.168.5.130/26.

W6.2 Explain TCP congestion control (slow start, congestion avoidance, fast retransmit / recovery).

W6.3 Compare symmetric and asymmetric encryption and explain briefly how HTTPS (TLS) uses both. Include an RSA toy example.

W6.4 Explain SQL injection, XSS and CSRF, and how to prevent each.

W6.5 Compare distance-vector and link-state routing. What is the count-to-infinity problem?

Topic 6: Answers and Explanations

MCQ answer key

- 6.1 Answer B.** $32 - 23 = 9$ host bits: $2^9 - 2 = 510$ usable addresses.
- 6.2 Answer C.** ARP broadcasts "who has this IP?" and the owner replies with its MAC address.
- 6.3 Answer B.** $C = 3000 \log_2(1 + 1000) = 3000 \times 9.97 = \text{about } 29.9 \text{ kbps.}$
- 6.4 Answer B.** Client SYN, server SYN+ACK, client ACK.
- 6.5 Answer B.** $S = G e^{-(2G)}$, maximised at $G = 0.5$ giving $1/(2e) = 18.4\%$. Slotted ALOHA doubles it to 36.8%.
- 6.6 Answer B.** The number of appended zeros equals the degree of the generator polynomial: 3 (the remainder is the 3-bit CRC).
- 6.7 Answer B.** $2^r \geq m + r + 1$: $r = 3$ gives $8 \geq 11$ (false); $r = 4$ gives $16 \geq 12$ (true). So 4.
- 6.8 Answer C.** SR needs window $\leq 2^{(n-1)} = 4$ so that old and new windows never overlap. (Go-Back-N allows 7.)
- 6.9 Answer B.** A query and answer normally fit one small datagram, and skipping the handshake keeps lookups fast. TCP is used for zone transfers or truncated (large) responses.
- 6.10 Answer B.** Only the sender holds the private key; anyone can verify with the sender's public key. This gives authentication and non-repudiation.

Model answers (written)

W6.1 Divide 192.168.1.0/24 into 4 equal subnets. Give network, usable range and broadcast for each. Then find the network and broadcast of 192.168.5.130/26.

Four equal subnets need 2 extra bits: /26, block size 64, 62 hosts each.

Subnet	Network	Usable range	Broadcast
1	192.168.1.0/26	.1 - .62	.63
2	192.168.1.64/26	.65 - .126	.127
3	192.168.1.128/26	.129 - .190	.191
4	192.168.1.192/26	.193 - .254	.255

For 192.168.5.130/26 the block is 128-191, so network = **192.168.5.128**, broadcast = **192.168.5.191**, hosts .129 to .190. VLSM lets different subnets use different prefix lengths to save addresses.

W6.2 Explain TCP congestion control (slow start, congestion avoidance, fast retransmit / recovery).

- **Slow start:** cwnd starts at 1 MSS and doubles each RTT (exponential) until it reaches ssthresh.
- **Congestion avoidance (AIMD):** above ssthresh cwnd grows by 1 MSS per RTT (additive increase). On loss it is cut multiplicatively.
- **Timeout:** ssthresh = cwnd/2, cwnd = 1 MSS, restart slow start.
- **Fast retransmit / recovery:** 3 duplicate ACKs signal a lost segment; retransmit immediately, set ssthresh = cwnd/2 and continue from cwnd = ssthresh (Reno) instead of restarting from 1.
- Example: ssthresh 16, loss at cwnd = 20 by timeout: new ssthresh = 10, cwnd = 1, then 1, 2, 4, 8, 10, 11 ...

W6.3 Compare symmetric and asymmetric encryption and explain briefly how HTTPS (TLS) uses both. Include an RSA toy example.

- **Symmetric (AES):** one shared key, fast, key distribution problem. **Asymmetric (RSA, ECC):** public/private pair, slow, solves key distribution and enables signatures.
- **TLS (simplified):** client hello -> server sends its certificate (public key signed by a CA) -> client verifies the certificate chain -> they agree a session key using asymmetric crypto (key exchange) -> all data is encrypted with the fast symmetric session key and protected with a MAC/AEAD.
- **RSA toy:** $p = 3, q = 11$: $n = 33, \phi = 20$; choose $e = 3, d = 7$ ($3 \times 7 = 21 = 1 \bmod 20$). Encrypt $m = 4$: $4^3 \bmod 33 = 31$. Decrypt: $31^7 \bmod 33 = 4$.

W6.4 Explain SQL injection, XSS and CSRF, and how to prevent each.

- **SQL injection:** user input is concatenated into a query (name = "" OR '1'='1"). Prevent with parameterised queries/prepared statements, input validation, least-privilege DB accounts.
- **XSS:** attacker-supplied script is stored or reflected in a page and runs in other users' browsers (steals cookies/sessions). Prevent with output encoding/escaping, Content-Security-Policy, HttpOnly cookies.

- **CSRF:** a malicious site makes the victim's browser send an authenticated request to another site. Prevent with anti-CSRF tokens, SameSite cookies, re-authentication for sensitive actions.
- Related network attacks: ARP spoofing (use dynamic ARP inspection), DHCP starvation (DHCP snooping, port security), SYN flood (SYN cookies).

W6.5 Compare distance-vector and link-state routing. What is the count-to-infinity problem?

Aspect	Distance vector	Link state
Knowledge	Only neighbours' distance vectors	Full topology map from flooded LSAs
Algorithm	Bellman-Ford	Dijkstra
Convergence	Slow, loops possible	Fast, more CPU/memory
Example	RIP	OSPF, IS-IS

- **Count to infinity:** if a link fails, routers keep advertising stale routes through each other and slowly increase the metric (bad news travels slowly). Mitigations: split horizon, poison reverse, hold-down timers, a small "infinity" (RIP uses 16).

Topic 7: Software Engineering and Information System Design

Key facts to revise

- **Process models:** waterfall (sequential, stable requirements), incremental, spiral (risk-driven), V-model (testing mapped to phases), prototyping, agile (Scrum: sprint, product backlog, daily stand-up, review, retrospective; XP practices such as pair programming and TDD).
- **Requirements:** functional (what the system does) vs non-functional (performance, security, usability, reliability); SRS document; use cases and user stories; traceability.
- **UML:** use case (actors and goals), class (structure: attributes, methods, associations with multiplicity, aggregation, composition, inheritance), sequence (time-ordered messages), activity (workflow), state machine (object life cycle), component and deployment diagrams.
- **Design principles:** high cohesion and low coupling (worst coupling = content, best = data; best cohesion = functional); SOLID (Single responsibility, Open/closed, Liskov substitution, Interface segregation, Dependency inversion).
- **Design patterns:** creational (Singleton, Factory Method, Abstract Factory, Builder, Prototype), structural (Adapter, Decorator, Facade, Proxy, Composite), behavioural (Observer, Strategy, Command, State, Iterator, Template Method).
- **Testing:** black box (equivalence partitioning, boundary value analysis) vs white box (statement, branch, condition, path coverage; cyclomatic complexity $V(G) = E - N + 2P$). Levels: unit, integration, system, acceptance. Regression testing after changes.
- **Estimation:** COCOMO basic effort = $a(KLOC)^b$ (organic $a = 2.4, b = 1.05$; semi-detached 3.0, 1.12; embedded 3.6, 1.20); function points; PERT/CPM for schedules (critical path = longest path).
- **Maintenance:** corrective, adaptive, perfective, preventive. **Information systems:** ER modelling -> relational schema, DFD levels (context diagram, level 1...), normalisation, CRUD matrices, system analysis vs design.

Part A: MCQs (attempt first, answers follow Part B)

7.1 Which process model is best when requirements are uncertain and risk management matters most?

- | | |
|--------------|-------------|
| A) Waterfall | B) Spiral |
| C) V-model | D) Big bang |

7.2 Which type of coupling is the worst (tightest)?

- | | |
|--------------------|---------------------|
| A) Data coupling | B) Stamp coupling |
| C) Common coupling | D) Content coupling |

7.3 The Observer pattern is classified as:

- | | |
|----------------|------------------|
| A) Creational | B) Structural |
| C) Behavioural | D) Architectural |

7.4 An input accepts integers 1 to 100. Basic boundary value analysis (min, min+, nominal, max-, max) picks:

- | | |
|-----------------------|----------------------|
| A) 0, 1, 50, 100, 101 | B) 1, 2, 50, 99, 100 |
| C) 1, 50, 100 | D) 0, 50, 101 |

7.5 A control-flow graph has $E = 12$ edges, $N = 9$ nodes, one connected component. Cyclomatic complexity:

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 6 |

7.6 Basic COCOMO (organic: effort = $2.4 \times KLOC^{1.05}$) for 32 KLOC gives about:

- | | |
|----------------------|----------------------|
| A) 76 person-months | B) 91 person-months |
| C) 120 person-months | D) 154 person-months |

7.7 Which UML diagram shows time-ordered message exchange between objects?

- | | |
|------------------|-----------------------|
| A) Class diagram | B) Sequence diagram |
| C) State diagram | D) Deployment diagram |

7.8 In SOLID, the letter S stands for:

- A) Single Responsibility
- B) Substitution
- C) Segregation
- D) Singleton

7.9 In Scrum, the fixed time-boxed iteration is called a:

- A) Sprint
- B) Epic
- C) Spike
- D) Release

7.10 Which of these is a non-functional requirement?

- A) A user can reset a password
- B) The system must respond within 2 seconds
- C) An admin can delete users
- D) The cart shows the item total

Part B: Written questions (about 10 marks each)

W7.1 Compare the Waterfall and Agile (Scrum) approaches. When would you choose each?

W7.2 Explain the SOLID principles with a one-line example each.

W7.3 Describe an ATM cash withdrawal using a use case and a sequence diagram (in words).

W7.4 Explain statement coverage and branch coverage with an example. Which is stronger?

W7.5 Write a thread-safe Singleton in Java and explain the Observer pattern.

W7.6 Explain DFD levels and how an ER diagram is converted to relational tables.

Topic 7: Answers and Explanations

MCQ answer key

7.1 Answer B. The spiral model repeats planning, risk analysis, engineering and evaluation in each loop.

7.2 Answer D. Content coupling: one module modifies or depends on the internals of another. Data coupling is the best.

7.3 Answer C. Observer defines a one-to-many dependency so subscribers are notified of state changes. It concerns object interaction (behaviour).

7.4 Answer B. Basic BVA tests the boundaries and one step inside them plus a nominal value. (Robust BVA adds 0 and 101.)

7.5 Answer C. $V(G) = E - N + 2P = 12 - 9 + 2 = 5$.

7.6 Answer B. $32^{1.05} = 2^{5.25} = 38.05$; $2.4 \times 38.05 = 91.3$.

7.7 Answer B. Sequence diagrams have lifelines and messages ordered vertically in time.

7.8 Answer A. A class should have only one reason to change.

7.9 Answer A. A sprint typically lasts 1 to 4 weeks and ends with a potentially shippable increment.

7.10 Answer B. Performance is a quality attribute (non-functional); the others describe features.

Model answers (written)

W7.1 Compare the Waterfall and Agile (Scrum) approaches. When would you choose each?

Aspect	Waterfall	Agile / Scrum
Structure	Sequential phases with sign-off	Short iterations (sprints) delivering working increments
Requirements	Fixed up front	Expected to change; backlog re-prioritised
Customer	Involved at start and end	Continuous collaboration
Risk	Found late (integration/testing)	Found early through frequent feedback
Best for	Stable, well-understood, regulated projects	Uncertain, fast-changing products

Agile costs: less predictable scope/cost, needs disciplined teams; waterfall costs: change is expensive and users see the product late.

W7.2 Explain the SOLID principles with a one-line example each.

- **S** Single Responsibility: a class has one reason to change (separate Invoice from InvoicePrinter).
- **O** Open/Closed: open for extension, closed for modification (add a new payment type by adding a class, not editing a giant if-else).
- **L** Liskov Substitution: subclasses must be usable wherever the base type is expected (a Square that breaks Rectangle's behaviour violates it).
- **I** Interface Segregation: many small interfaces beat one fat one (Printer vs Scanner instead of MultiFunctionDevice).
- **D** Dependency Inversion: depend on abstractions, not concrete classes (inject a Repository interface into a service).

W7.3 Describe an ATM cash withdrawal using a use case and a sequence diagram (in words).

- **Actors:** Customer, Bank system. **Use case Withdraw Cash** includes Authenticate (PIN); extends with Print Receipt.
- **Sequence:** Customer -> ATM: insert card, enter PIN. ATM -> Bank: verifyPIN(card, pin). Bank -> ATM: valid. Customer -> ATM: request amount. ATM -> Bank: authorise(amount). Bank -> ATM: approved (balance debited). ATM -> Customer: dispense cash, eject card. Alternative flows: wrong PIN (three tries, retain card), insufficient balance (message, no dispense), ATM out of cash.
- **Lifelines:** Customer, ATM, Bank; activation bars show processing; alt/opt fragments show conditions.

W7.4 Explain statement coverage and branch coverage with an example. Which is stronger?

```
if (a > 0) x = 1;      // decision 1
if (b > 0) y = 1;      // decision 2
```

- **Statement coverage:** every statement executes at least once: one test ($a > 0$, $b > 0$) suffices.
- **Branch coverage:** every decision takes both true and false outcomes: needs at least two tests, e.g. ($a > 0$, $b > 0$) and ($a \leq 0$, $b \leq 0$).
- **Branch coverage is stronger:** it subsumes statement coverage (100% branch implies 100% statement) but not the reverse. Condition and path coverage are stronger still. Cyclomatic complexity gives the number of independent paths (here 3).

W7.5 Write a thread-safe Singleton in Java and explain the Observer pattern.

```
public final class Config {
    private static volatile Config instance;
    private Config() {}
    public static Config getInstance() {
        if (instance == null) {
            synchronized (Config.class) {
                if (instance == null) instance = new Config();
            }
        }
        return instance;
    }
}
```

Singleton: private constructor + one shared instance (double-checked locking; volatile prevents reordering). Simpler alternative: an enum singleton.

Observer: a Subject keeps a list of Observers and calls update() on each when its state changes (publish/subscribe; used in GUIs and event systems). It gives loose coupling between the subject and its dependents.

W7.6 Explain DFD levels and how an ER diagram is converted to relational tables.

- **DFD:** level 0 (context diagram) shows the whole system as one process with external entities; level 1 splits it into major processes, data stores and flows; deeper levels decompose further. Rules: balance parent/child flows, no flow directly between two entities or two stores.
- **ER to tables:** strong entity -> table with primary key; 1:N relationship -> foreign key on the N side; M:N -> junction table with both keys (plus relationship attributes); multivalued attribute -> separate table; weak entity -> table with the owner key as part of its key; 1:1 -> foreign key on either side; ISA hierarchy -> table per entity or one combined table.
- Afterwards normalise to at least 3NF.

Topic 8: Digital Logic Design, Microprocessors and Microcontrollers

Key facts to revise

- **Boolean laws:** De Morgan $(AB)' = A' + B'$, $(A+B)' = A'B'$; absorption $A + AB = A$; consensus $AB + A'C + BC = AB + A'C$. NAND and NOR are universal: NOT = tie the inputs; AND = NAND then NOT.
- **K-maps:** group 1s in powers of two (1, 2, 4, 8); larger groups remove more variables; groups wrap around edges; don't-cares may join groups. SOP from 1s, POS from 0s.
- **Combinational blocks:** half adder ($S = A \oplus B$, $C = AB$); full adder ($S = A \oplus B \oplus C_{in}$, $C_{out} = AB + C_{in}(A \oplus B)$); n -to- 2^n decoder; 2^n -to-1 mux needs n select lines (any function of $n+1$ variables can be built from a 2^n -to-1 mux); encoder; comparator; ripple-carry vs carry-lookahead adder.
- **Sequential:** SR, D ($Q^+ = D$), JK ($Q^+ = JQ' + K'Q$; $J=K=1$ toggles), T ($Q^+ = T \oplus Q$). A mod- N counter needs $\lceil \log_2 N \rceil$ flip-flops. Mealy outputs depend on state and input; Moore outputs depend on state only.
- **Number systems:** two's complement = invert + 1 (range $-2^{(n-1)}$ to $2^{(n-1)} - 1$); BCD encodes each decimal digit in 4 bits; Gray code changes one bit at a time ($g_i = b_i \oplus b_{i+1}$). Memory of 2^n words needs n address lines.
- **8085:** 8-bit data bus, 16-bit address bus (64 KB), flags S, Z, AC, P, CY, interrupts TRAP, RST7.5, RST6.5, RST5.5, INTR.
- **8086:** 16-bit, 20-bit address bus (1 MB), BIU + EU with a 6-byte prefetch queue, segment registers CS, DS, SS, ES; physical address = segment $\times$ 16 + offset; flags include CF, PF, AF, ZF, SF, OF, DF, IF, TF.
- **8051 microcontroller:** 8-bit, 4 KB on-chip ROM, 128 bytes RAM, 32 I/O lines in 4 ports, two 16-bit timers, one full-duplex UART, 5 interrupt sources. Von Neumann (8086) vs Harvard (8051) memory model.

Part A: MCQs (attempt first, answers follow Part B)

8.1 A full adder has $A = B = C_{in} = 1$. Its outputs are:

- | | |
|-------------------|-------------------|
| A) Sum 0, Carry 1 | B) Sum 1, Carry 1 |
| C) Sum 1, Carry 0 | D) Sum 0, Carry 0 |

8.2 How many select lines does a 4-to-1 multiplexer need?

- | | |
|------|------|
| A) 1 | B) 2 |
| C) 3 | D) 4 |

8.3 Minimum number of flip-flops for a mod-10 (decade) counter:

- | | |
|------|-------|
| A) 3 | B) 4 |
| C) 5 | D) 10 |

8.4 Binary 101101 converted to Gray code:

- | | |
|-----------|-----------|
| A) 111011 | B) 110110 |
| C) 101101 | D) 011011 |

8.5 Address lines needed for an 8K x 8 memory:

- | | |
|-------|-------|
| A) 8 | B) 10 |
| C) 13 | D) 16 |

8.6 Decimal 25 in BCD is:

- | | |
|-------------|-------------|
| A) 11001 | B) 00100101 |
| C) 00011001 | D) 00101000 |

8.7 A T flip-flop with $T = 1$ used as a frequency divider divides the clock by:

- | | |
|------|------|
| A) 1 | B) 2 |
| C) 4 | D) 8 |

8.8 The simplified form of $F(A,B,C,D) = \text{sum of minterms } (0, 2, 8, 10)$ is:

- | | |
|-----------|---------|
| A) $B'D'$ | B) BD |
| C) $A'C'$ | D) AC |

8.9 The address bus of the 8085 microprocessor is:

- | | |
|-------------------|-------------------|
| A) 8 bits | B) 16 bits |
| C) 20 bits | D) 32 bits |

8.10 8086: CS = 1000h and IP = 0234h. The physical address of the next instruction is:

- | | |
|------------------|------------------|
| A) 10234h | B) 1234h |
| C) 12340h | D) 10023h |

Part B: Written questions (about 10 marks each)

W8.1 Design a 3-bit synchronous up counter using T flip-flops.

W8.2 Minimise $F(A,B,C)$ = sum of minterms (1, 3, 4, 5, 7) using a K-map and implement it with NAND gates only.

W8.3 Build a full adder from two half adders and an OR gate. Compare a ripple-carry adder with carry lookahead.

W8.4 Design a Mealy sequence detector for the overlapping sequence 101.

W8.5 Describe the 8086 architecture (BIU, EU, segmentation) and list the main 8051 features.

Topic 8: Answers and Explanations

MCQ answer key

- 8.1 Answer B.** $1 + 1 + 1 = 3 = \text{binary } 11$: sum bit 1, carry 1.
- 8.2 Answer B.** $2^2 = 4$ inputs, so 2 select lines.
- 8.3 Answer B.** $2^3 = 8 < 10 \leq 2^4 = 16$, so 4 flip-flops.
- 8.4 Answer A.** MSB stays 1; each next bit = XOR of adjacent binary bits: 1, $1^0=1$, $0^1=1$, $1^1=0$, $1^0=1$, $0^1=1 \rightarrow 111011$.
- 8.5 Answer C.** $8K = 2^{13}$ words, so 13 address lines (and 8 data lines).
- 8.6 Answer B.** Each digit in 4 bits: 2 = 0010, 5 = 0101.
- 8.7 Answer B.** It toggles on every clock edge, so Q has half the clock frequency.
- 8.8 Answer A.** Minterms 0000, 0010, 1000, 1010 all have B = 0 and D = 0 (they form a corner quad), so $F = B'D'$.
- 8.9 Answer B.** 16 address lines give $2^{16} = 64$ KB of memory.
- 8.10 Answer A.** Physical = $1000h \times 16 + 0234h = 10000h + 0234h = 10234h$.

Model answers (written)

W8.1 Design a 3-bit synchronous up counter using T flip-flops.

- States 000 $\rightarrow$ 001 $\rightarrow$... $\rightarrow$ 111 $\rightarrow$ 000. A T flip-flop toggles when T = 1.
- Q0 toggles every clock: **T0 = 1**. Q1 toggles when Q0 = 1: **T1 = Q0**. Q2 toggles when Q1 and Q0 are both 1: **T2 = Q1.Q0**.
- All flip-flops share one clock (synchronous), so there are no ripple delays. For a ripple (asynchronous) counter, chain each flip-flop clock from the previous output instead.
- For a mod-N counter, add logic that detects state N and resets (or design the next-state table directly).

W8.2 Minimise $F(A,B,C)$ = sum of minterms (1, 3, 4, 5, 7) using a K-map and implement it with NAND gates only.

- Minterms: 001, 011, 100, 101, 111. Quad {1,3,5,7} (C = 1) gives **C**. Pair {4,5} (A = 1, B = 0) gives **AB'**.
- $F = C + AB'$** .
- NAND-only: $F = (C' \cdot (AB')')$. Implement with: $B' = \text{NAND}(B,B)$; $X = \text{NAND}(A, B')$ (this is $(AB')'$); $C' = \text{NAND}(C,C)$; $F = \text{NAND}(C', X)$. That is 4 NAND gates.

W8.3 Build a full adder from two half adders and an OR gate. Compare a ripple-carry adder with carry lookahead.

- HA1 adds A and B: $s1 = A \oplus B$, $c1 = AB$. HA2 adds s1 and Cin: $\text{Sum} = s1 \oplus \text{Cin}$, $c2 = s1 \cdot \text{Cin}$. **$\text{Cout} = c1 + c2$** .
- Ripple-carry:** n full adders chained; delay grows linearly with n because each carry waits for the previous stage.
- Carry lookahead:** compute generate $G_i = A_i B_i$ and propagate $P_i = A_i \oplus B_i$; carries come directly: $C1 = G0 + P0 C0$, $C2 = G1 + P1 G0 + P1 P0 C0$, ... So all carries are formed in a constant number of gate levels at the cost of more hardware.

W8.4 Design a Mealy sequence detector for the overlapping sequence 101.

States: S0 (nothing useful seen), S1 (seen 1), S2 (seen 10). Output Z = 1 only on the transition that completes 101.

State	Input 0	Input 1
S0	S0 / 0	S1 / 0
S1	S2 / 0	S1 / 0
S2	S0 / 0	S1 / 1

- On S2 with input 1 the pattern 101 is complete, output 1, and the final 1 also starts a new pattern, so go to S1 (overlap).
- A Moore machine needs one extra state (S3 = "just detected") because its output depends on the state only.

W8.5 Describe the 8086 architecture (BIU, EU, segmentation) and list the main 8051 features.

- **8086:** two units working in parallel. The Bus Interface Unit (BIU) fetches instructions into a 6-byte queue and computes physical addresses; the Execution Unit (EU) decodes and executes them (ALU, general registers AX, BX, CX, DX, pointers SP, BP, SI, DI). Prefetching overlaps fetch and execute (a simple pipeline).
- **Segmentation:** four 64 KB segments (code, data, stack, extra) selected by CS, DS, SS, ES; physical address = segment $\times 16$ + offset (20 bits, 1 MB). Example: 2000h:0050h $\rightarrow$ 20050h.
- **8051:** 8-bit MCU with 4 KB ROM, 128 B RAM, 4 ports (32 I/O pins), two 16-bit timers/counters, serial UART, five interrupts, Harvard-style separate code/data spaces. A microcontroller integrates memory and peripherals on one chip; a microprocessor needs external memory and I/O chips.
- **Stack push/pop with a simple ISA (pattern):** PUSH x = decrement SP then store x at [SP]; POP x = load x from [SP] then increment SP (adapt to the instructions given).

Topic 9: Computer Architecture and Operating Systems

Key facts to revise

- **Performance:** CPU time = IC x CPI x clock period; Amdahl: speedup = $1 / ((1 - f) + f/s)$; MIPS = clock / (CPI x 10^6). Pipeline speedup for k stages and n instructions = $nk / (k + n - 1)$, approaching k.
- **Hazards:** structural (resource conflict), data (RAW, WAR, WAW; fix with forwarding/stalls), control (branches; fix with prediction/delay slots). Load-use hazards need one stall even with forwarding.
- **Cache:** address = tag | index | offset; offset bits = $\log_2(\text{block size})$; index bits = $\log_2(\text{number of sets})$; average memory access time = hit time + miss rate x miss penalty. Mapping: direct (1 line per set), set-associative, fully associative. Write-through vs write-back.
- **Virtual memory:** page table entries = $2^{(\text{VA bits} - \text{page offset bits})}$; TLB caches translations; EAT with TLB = $h(t + m) + (1 - h)(t + 2m)$; with page faults EAT = $(1 - p)m + p \times \text{fault time}$; thrashing when the working set exceeds memory.
- **Scheduling:** FCFS (convoy effect), SJF/SRTF (optimal average waiting, starvation), Round Robin (time quantum), priority (aging fixes starvation). Turnaround = completion - arrival; waiting = turnaround - burst.
- **Synchronisation:** critical-section conditions: mutual exclusion, progress, bounded waiting. Semaphores wait/P and signal/V; classic problems: producer-consumer, readers-writers, dining philosophers. Race condition = outcome depends on interleaving.
- **Deadlock:** four necessary conditions (mutual exclusion, hold and wait, no preemption, circular wait). Prevent by breaking one, avoid with Banker's algorithm (safe state), detect with a resource-allocation graph. With n processes each needing at most m instances, deadlock cannot occur if instances $\geq n(m - 1) + 1$.
- **Memory and disk:** paging causes internal fragmentation, segmentation causes external; page replacement FIFO (Belady's anomaly), LRU, Optimal; disk scheduling FCFS, SSTF, SCAN (elevator), C-SCAN; RAID 0/1/5/6.

Part A: MCQs (attempt first, answers follow Part B)

9.1 40% of a program can be made 4 times faster. Overall speedup (Amdahl):

- | | |
|---------|---------|
| A) 1.25 | B) 1.43 |
| C) 2.0 | D) 4.0 |

9.2 A program has 2×10^9 instructions, CPI = 1.5, clock 2 GHz. CPU time:

- | | |
|-----------|--------|
| A) 1.5 s | B) 3 s |
| C) 0.75 s | D) 6 s |

9.3 32 KB direct-mapped cache, 64-byte blocks, 32-bit addresses. Number of tag bits:

- | | |
|-------|-------|
| A) 15 | B) 17 |
| C) 19 | D) 21 |

9.4 32-bit virtual address, 4 KB pages. Number of page table entries:

- | | |
|-------------|-------------|
| A) 2^{10} | B) 2^{12} |
| C) 2^{20} | D) 2^{32} |

9.5 Round Robin, quantum 2, all arrive at 0: P1 burst 5, P2 burst 3, P3 burst 1. Average waiting time:

- | | |
|---------|---------|
| A) 3.33 | B) 4.33 |
| C) 5 | D) 6 |

9.6 Disk head at 53; requests 98, 183, 37, 122, 14, 124, 65, 67. Total head movement with FCFS:

- | | |
|--------|--------|
| A) 236 | B) 640 |
| C) 500 | D) 320 |

9.7 A counting semaphore starts at 2. Three wait() and one signal() operations are performed. Its final value (classic definition):

- | | |
|-------|------|
| A) 0 | B) 1 |
| C) -1 | D) 2 |

9.8 Memory access 100 ns, page fault service 8 ms, fault rate 10^{-6} . Effective access time:

- | | |
|-----------------|------------------|
| A) about 100 ns | B) about 108 ns |
| C) about 200 ns | D) about 8000 ns |

9.9 3 processes each need up to 2 instances of a resource. Minimum instances that guarantee no deadlock:

- | | |
|------|------|
| A) 3 | B) 4 |
| C) 5 | D) 6 |

9.10 Paging suffers from:

- | | |
|--------------------------------|--|
| A) External fragmentation only | B) Internal fragmentation |
| C) No fragmentation | D) Neither internal nor external fragmentation |

Part B: Written questions (about 10 marks each)

W9.1 Explain pipeline hazards and how they are resolved.

W9.2 Write the producer-consumer solution using semaphores.

W9.3 Explain paging and the TLB. Compute the EAT with a 90% TLB hit rate, TLB access 10 ns, memory access 100 ns.

W9.4 Banker's algorithm example: 10 instances of a resource. Allocated: P1 = 3, P2 = 2, P3 = 2. Maximum need: P1 = 9, P2 = 4, P3 = 7. Is the state safe?

W9.5 Compare disk scheduling algorithms using head at 53 and queue 98, 183, 37, 122, 14, 124, 65, 67 (cylinders 0-199).

Topic 9: Answers and Explanations

MCQ answer key

9.1 Answer B. $1 / (0.6 + 0.4/4) = 1 / 0.7 = 1.43$.

9.2 Answer A. $IC \times CPI / f = 2e9 \times 1.5 / 2e9 = 1.5$ s.

9.3 Answer B. Offset = 6 bits. Blocks = $32K/64 = 512$, so index = 9 bits. Tag = $32 - 9 - 6 = 17$.

9.4 Answer C. Offset = 12 bits, so the page number has 20 bits: 2^{20} entries.

9.5 Answer B. Order P1,P2,P3,P1,P2,P1: completions P3=5, P2=8, P1=9. Waiting = completion - burst: P1 = 4, P2 = 5, P3 = 4. Average = $13/3 = 4.33$.

9.6 Answer B. $45 + 85 + 146 + 85 + 108 + 110 + 59 + 2 = 640$ cylinders. (SSTF gives 236.)

9.7 Answer A. $2 \rightarrow 1 \rightarrow 0 \rightarrow -1$ (third waiter blocks) $\rightarrow$ signal makes it 0 and wakes the waiter.

9.8 Answer B. $EAT = (1 - 10^{-6}) \times 100 + 10^{-6} \times 8,000,000 = \text{about } 100 + 8 = 108$ ns.

9.9 Answer B. $n(m - 1) + 1 = 3 \times 1 + 1 = 4$. With 3 instances each process could hold one and wait for another.

9.10 Answer B. The last page of a process is usually partly empty (internal). Segmentation suffers from external fragmentation.

Model answers (written)

W9.1 Explain pipeline hazards and how they are resolved.

- **Structural:** two instructions need the same hardware (e.g. one memory port). Fix: duplicate resources (separate instruction and data caches), or stall.
- **Data (RAW, WAR, WAW):** an instruction needs a result not yet written. Fix: forwarding/bypassing from later pipeline stages, compiler scheduling, or stall cycles (a load followed by a dependent instruction still needs one bubble).
- **Control:** the branch outcome is unknown when the next instruction is fetched. Fix: branch prediction (static/dynamic), delayed branches, flush on misprediction.
- Ideal CPI = 1; real CPI = 1 + stall cycles per instruction.

W9.2 Write the producer-consumer solution using semaphores.

```
semaphore mutex = 1;    // protects the buffer
semaphore empty = N;    // free slots
semaphore full = 0;     // filled slots

Producer:               Consumer:
while (true) {          while (true) {
    item = produce();    wait(full);
    wait(empty);         wait(mutex);
    wait(mutex);         item = remove_from_buffer();
    insert_into_buffer(item);
    signal(mutex);       signal(mutex);
    signal(full);        signal(empty);
                        consume(item);
}                        }
```

The order of wait(empty)/wait(full) before wait(mutex) matters: reversing it can cause deadlock (a producer holds the mutex while waiting for space). Semaphores enforce mutual exclusion (mutex) and synchronisation (empty/full).

W9.3 Explain paging and the TLB. Compute the EAT with a 90% TLB hit rate, TLB access 10 ns, memory access 100 ns.

- Paging splits the virtual address into (page number, offset). The page table maps pages to frames; a TLB caches recent translations so most accesses avoid the extra memory reference for the page table.
- **Hit (90%):** $10 + 100 = 110$ ns. **Miss (10%):** $10 + 100$ (page table) + 100 (data) = 210 ns.
- **EAT = $0.9 \times 110 + 0.1 \times 210 = 99 + 21 = 120$ ns.**
- Larger page tables use multi-level or inverted page tables. A page fault (page not in memory) triggers the OS to load it from disk, which dominates cost when the fault rate is not tiny.

W9.4 Banker's algorithm example: 10 instances of a resource. Allocated: P1 = 3, P2 = 2, P3 = 2. Maximum need: P1 = 9, P2 = 4, P3 = 7. Is the state safe?

- Available = $10 - 7 = 3$. Need = Max - Allocated: P1 = 6, P2 = 2, P3 = 5.
- P2 (need $2 \leq 3$) runs and finishes, releasing 4 in total: available = $3 - 2 + 4 = 5$.

- P3 (need 5 $\leq$ 5) runs and finishes: available = 5 - 5 + 7 = 7.
- P1 (need 6 $\leq$ 7) finishes.
- Safe sequence **P2, P3, P1**, so the state is safe. A request is granted only if the resulting state is still safe.

W9.5 Compare disk scheduling algorithms using head at 53 and queue 98, 183, 37, 122, 14, 124, 65, 67 (cylinders 0-199).

- **FCFS:** 53 -> 98 -> 183 -> 37 -> 122 -> 14 -> 124 -> 65 -> 67: total **640**.
- **SSTF** (nearest first): 53 -> 65 -> 67 -> 37 -> 14 -> 98 -> 122 -> 124 -> 183: 12 + 2 + 30 + 23 + 84 + 24 + 2 + 59 = **236**. Can starve far requests.
- **SCAN (elevator):** moving toward 0 first: 53 -> 37 -> 14 -> 0 -> 65 -> 67 -> 98 -> 122 -> 124 -> 183: 53 + 183 = 236 (to the end and back).
- **C-SCAN:** serve in one direction only and jump back, giving more uniform waiting times.

Topic 10: Artificial Intelligence and Machine Learning

Key facts to revise

- **Uninformed search:** BFS (complete; optimal for equal step costs; $O(b^d)$ time and space), DFS ($O(bm)$ space, not optimal, may not terminate), UCS (optimal for positive costs), depth-limited, IDDFS (complete, optimal for unit costs, $O(bd)$ space).
- **Informed search:** greedy best-first $f = h$; $A^* f = g + h$, optimal if h is admissible (never overestimates) and, for graph search, consistent ($h(n) \leq c(n, n') + h(n')$). Hill climbing can get stuck at local maxima; simulated annealing and genetic algorithms escape.
- **Adversarial:** minimax; alpha-beta pruning gives the same result, best-case $O(b^{d/2})$.
- **CSP:** backtracking with MRV, degree heuristic, forward checking, arc consistency (AC-3).
- **Logic and probability:** resolution, unification; Bayes rule; Naive Bayes assumes features are conditionally independent given the class.
- **Supervised learning:** linear/logistic regression, k-NN (small k = low bias, high variance), decision trees (entropy, information gain), SVM, Naive Bayes, neural networks. Unsupervised: k-means (minimises within-cluster sum of squares), hierarchical clustering, PCA.
- **Evaluation:** accuracy, precision = $TP/(TP+FP)$, recall = $TP/(TP+FN)$, $F1 = 2PR/(P+R)$; cross-validation; overfitting vs underfitting; regularisation (L1, L2), dropout, early stopping.
- **Neural nets:** a single perceptron only learns linearly separable functions (not XOR); activation functions (sigmoid in $(0,1)$, tanh in $(-1,1)$, ReLU $\max(0,x)$); backpropagation uses the chain rule; gradient descent $w = w - \eta \times \text{gradient}$; CNNs for images, RNN/LSTM/transformers for sequences.
- **Reinforcement learning:** Q-learning update $Q(s,a) = Q(s,a) + \alpha [r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$.

Part A: MCQs (attempt first, answers follow Part B)

10.1 Which search is complete and optimal when all step costs are equal?

- | | |
|----------------------|------------------|
| A) DFS | B) BFS |
| C) Greedy best-first | D) Hill climbing |

10.2 Greedy best-first search orders nodes using:

- | | |
|------------------|------------------|
| A) $g(n)$ | B) $h(n)$ |
| C) $g(n) + h(n)$ | D) $g(n) - h(n)$ |

10.3 A single-layer perceptron cannot learn:

- | | |
|--------|--------|
| A) AND | B) OR |
| C) XOR | D) NOT |

10.4 k-means clustering minimises:

- | | |
|------------------------------|--|
| A) Distance between clusters | B) Within-cluster sum of squared distances |
| C) Entropy | D) Misclassification rate |

10.5 Naive Bayes assumes that:

- | | |
|---|--|
| A) Features are conditionally independent given the class | B) Features are independent of the class |
| C) All classes are equally likely | D) Features are Gaussian |

10.6 sigmoid(0) equals:

- | | |
|------|---------|
| A) 0 | B) 0.5 |
| C) 1 | D) 0.25 |

10.7 Increasing k in k-nearest neighbours generally:

- | | |
|---------------------------------|---------------------------------|
| A) Lowers bias, raises variance | B) Raises bias, lowers variance |
| C) Has no effect | D) Gives zero training error |

10.8 Entropy of a fair coin toss (in bits):

- | | |
|------|--------|
| A) 0 | B) 0.5 |
| C) 1 | D) 2 |

10.9 Best-case time complexity of alpha-beta pruning with perfect move ordering:

- | | |
|-------------|-------------------|
| A) $O(b^d)$ | B) $O(b^{(d/2)})$ |
| C) $O(bd)$ | D) $O(d)$ |

10.10 Q-learning: $Q(s,a) = 0.5$, $\alpha = 0.1$, $r = 1$, $\gamma = 0.9$, $\max Q(s',a') = 2$. New $Q(s,a)$:

- | | |
|---------|---------|
| A) 0.73 | B) 0.5 |
| C) 2.8 | D) 0.23 |

Part B: Written questions (about 10 marks each)

W10.1 Apply minimax with alpha-beta pruning: MAX root with three MIN children whose leaves are A: 3, 12, 8; B: 2, 4, 6; C: 14, 5, 2.

W10.2 Compare BFS, DFS, Uniform-Cost Search, IDDFS and A* (b = branching factor, d = solution depth, m = max depth).

W10.3 Explain the bias-variance trade-off. How do you detect and fix overfitting and underfitting?

W10.4 Explain gradient descent and backpropagation for a single sigmoid neuron with squared error. Compare batch, mini-batch and stochastic gradient descent.

W10.5 Build a decision tree split using information gain. Data: 14 examples (9 yes, 5 no). Outlook splits them into Sunny (2 yes, 3 no), Overcast (4 yes, 0 no), Rain (3 yes, 2 no). Compute the gain.

W10.6 Describe the k-means algorithm and its limitations.

Topic 10: Answers and Explanations

MCQ answer key

- 10.1 Answer B.** BFS expands nodes level by level, so the first goal found is the shallowest. DFS is neither complete (infinite depth) nor optimal.
- 10.2 Answer B.** Greedy uses only the heuristic estimate to the goal. A* uses $g + h$.
- 10.3 Answer C.** XOR is not linearly separable; a hidden layer is required.
- 10.4 Answer B.** Each iteration assigns points to the nearest centroid and recomputes centroids as means, reducing the within-cluster sum of squares.
- 10.5 Answer A.** That assumption lets the likelihood factorise into a product of per-feature probabilities.
- 10.6 Answer B.** $1 / (1 + e^0) = 1/2$. (Its derivative at 0 is 0.25.)
- 10.7 Answer B.** Larger k smooths the decision boundary (more bias, less variance). $k = 1$ gives zero training error and high variance.
- 10.8 Answer C.** $-(0.5 \log_2 0.5 + 0.5 \log_2 0.5) = 1$ bit, the maximum for two outcomes.
- 10.9 Answer B.** Perfect ordering lets the search look about twice as deep in the same time as plain minimax.
- 10.10 Answer A.** Target = $1 + 0.9 \times 2 = 2.8$. Error = $2.8 - 0.5 = 2.3$. New $Q = 0.5 + 0.1 \times 2.3 = 0.73$.

Model answers (written)

W10.1 Apply minimax with alpha-beta pruning: MAX root with three MIN children whose leaves are A: 3, 12, 8; B: 2, 4, 6; C: 14, 5, 2.

- MIN values: $A = \min(3, 12, 8) = 3$; $B = \min(2, 4, 6) = 2$; $C = \min(14, 5, 2) = 2$. **Root (MAX) = $\max(3, 2, 2) = 3$.**
- With alpha-beta: after A, $\alpha = 3$ at the root. In B the first leaf is $2 \leq \alpha$, so MIN's value is at most 2 and MAX will never choose B: **prune the leaves 4 and 6**. In C: 14 and 5 do not allow pruning yet; the leaf 2 gives $C = 2$. Result is the same value 3 with fewer nodes evaluated.
- α = best value MAX can guarantee so far; β = best value MIN can guarantee; prune when $\alpha \geq \beta$. Good move ordering prunes more.

W10.2 Compare BFS, DFS, Uniform-Cost Search, IDDFS and A* (b = branching factor, d = solution depth, m = max depth).

Algorithm	Complete	Optimal	Time	Space
BFS	Yes	Yes (unit cost)	$O(b^d)$	$O(b^d)$
DFS	No (infinite/loops)	No	$O(b^m)$	$O(bm)$
UCS	Yes (positive costs)	Yes	$O(b^{(1+C^*/e)})$	same as time
IDDFS	Yes	Yes (unit cost)	$O(b^d)$	$O(bd)$
A*	Yes	Yes (admissible h)	exponential in worst case	keeps all nodes

IDDFS combines BFS optimality with DFS memory; A* uses the heuristic to focus on promising nodes.

W10.3 Explain the bias-variance trade-off. How do you detect and fix overfitting and underfitting?

- Bias:** error from overly simple assumptions (underfitting). **Variance:** sensitivity to the particular training set (overfitting). Expected error = $\text{bias}^2 + \text{variance} + \text{irreducible noise}$; more model complexity lowers bias and raises variance.
- Detect:** high training and high validation error = underfitting; low training but high validation error = overfitting (use learning curves and k -fold cross-validation).
- Fix underfitting:** richer model/features, less regularisation, longer training. **Fix overfitting:** more data/augmentation, regularisation ($L1/L2$), simpler model, pruning, dropout, early stopping, ensembling.

W10.4 Explain gradient descent and backpropagation for a single sigmoid neuron with squared error. Compare batch, mini-batch and stochastic gradient descent.

- Neuron: $z = w \cdot x + b$, $y = \text{sigmoid}(z)$, loss $L = (1/2)(y - t)^2$.
- Chain rule: $dL/dw = (y - t) \times y(1 - y) \times x$. Update $w = w - \eta \times dL/dw$.
- Backpropagation applies the chain rule layer by layer from the output back to the inputs, reusing intermediate gradients.

- **Batch GD** uses all data per step (stable, slow); **SGD** uses one sample (noisy, fast, can escape shallow minima); **mini-batch** (e.g. 32-256 samples) is the practical compromise. Learning rate η too large diverges, too small is slow. Example: $w = 2$, gradient 0.5, $\eta = 0.1$ gives $w = 1.95$.

W10.5 Build a decision tree split using information gain. Data: 14 examples (9 yes, 5 no). Outlook splits them into Sunny (2 yes, 3 no), Overcast (4 yes, 0 no), Rain (3 yes, 2 no). Compute the gain.

- Entropy before: $H = -(9/14)\log_2(9/14) - (5/14)\log_2(5/14) = \mathbf{0.940}$.
- Sunny: $H(2,3) = 0.971$. Overcast: $H(4,0) = 0$. Rain: $H(3,2) = 0.971$.
- Weighted entropy after $= (5/14)(0.971) + (4/14)(0) + (5/14)(0.971) = 0.694$.
- **Gain = $0.940 - 0.694 = 0.246$** . Choose the attribute with the highest gain, then recurse on each branch; stop when a node is pure or attributes run out. Gain ratio and Gini are alternatives; prune the tree to reduce overfitting.

W10.6 Describe the k-means algorithm and its limitations.

- Choose k initial centroids; **assign** each point to its nearest centroid; **update** each centroid to the mean of its points; repeat until assignments stop changing. Each iteration costs $O(nkd)$ and the objective never increases.
- **Limitations:** k must be chosen (elbow/silhouette methods); sensitive to initialisation (use k-means++) and outliers; finds convex, similarly sized clusters; converges to a local optimum; needs numeric features (scaling matters).

Appendix A: Past-Paper Pattern Index

Themes below were seen in publicly listed BUET M.Sc. CSE papers (2019, 2021 online, 2024 and other sessions). They are summarised in my own words; question wording varies each year. Tick each one you can answer from memory.

Topic	Recurring patterns
Programming	C/C++ output puzzles (unsigned vs signed, pointer behaviour); Java string == vs equals; inner class creation; functional interface; reverse a string without a global.
Discrete / Prob.	Run-length and geometric probability; counting and recurrences; Bayes-style problems.
DSA	Loop detection in a linked list; $O(1)$ queue with a singly list; sparse vs dense graph representation; Bellman-Ford with negative weights; IDDFS vs DFS; sorting output tracing.
Compiler / TOC	Pumping lemma reasoning; NFA/DFA construction (odd 1s or even 0s); LL(1) table rules; CLR vs SLR vs LALR state counts; P, NP, NP-hard, NP-complete diagram; approximation vs heuristic; Turing machine statements.
Database	ACID; B+ tree leaf capacity; Cartesian product row counts; UPDATE with conditions (CASE); trigger writing; GROUP BY counting; sequence numbers / row identification.
Networks / Security	Subnet, host range and network address; half vs full duplex; DHCP starvation; UDP for DNS; valid host IDs for a /23 interface; socket API calls.
Software Eng.	Class and state diagrams for an online shop; maintenance life cycle; testing levels; Strategy and Factory patterns; reasons projects fail.
Digital Logic / uP	Minimum NAND/NOR realisation; XOR from a decoder and NOR; assembly for push/pop with a given tiny ISA.
Arch / OS	RAID levels and capacity; scheduling and page replacement; cache and memory calculations.
AI / ML	Overfitting and remedies; A* optimality conditions; state-space search comparisons.

Appendix B: One-Page Formula Sheet

- Circular permutations $(n-1)!$; stars and bars $C(n+k-1, k-1)$; $D(n) = (n-1)(D(n-1) + D(n-2))$; Bayes $P(A|B) = \frac{P(B|A)P(A)}{P(B)}$.
- Binomial mean np , variance npq ; sample variance = sum of squared deviations / $(n-1)$.
- Master theorem: compare $f(n)$ with $n^{\log_b a}$. Quick sort worst $O(n^2)$; merge sort $n \log n$; BFS/DFS $O(V+E)$; Dijkstra $O((V+E) \log V)$; Bellman-Ford $O(VE)$.
- Tree with n nodes has $n-1$ edges; K_n has $n(n-1)/2$ edges; number of labelled trees n^{n-2} ; Catalan $C_n = \frac{C(2n,n)}{(n+1)}$.
- Usable hosts = $2^{(32-p)} - 2$; Shannon $C = B \log_2(1+SNR)$; pure ALOHA 18.4%, slotted 36.8%; GBN window $2^n - 1$, SR window $2^{(n-1)}$; Hamming $2^r \geq m + r + 1$.
- CRC: append (degree of generator) zeros, then XOR-divide. RSA: $n = pq$, $\phi = (p-1)(q-1)$, $e \cdot d = 1 \pmod{\phi}$, $c = m^e \pmod{n}$.
- Cyclomatic $V(G) = E - N + 2P$; COCOMO organic effort = $2.4 (KLOC)^{1.05}$; F(sum) from K-map groups of 1, 2, 4, 8.
- Two's complement range $-2^{(n-1)}$ to $2^{(n-1)}-1$; JK: $Q+ = JQ' + K'Q$; T: $Q+ = T \text{ xor } Q$; memory 2^n words needs n address lines; 8086 physical = segment $\times 16$ + offset.
- CPU time = $IC \times CPI \times T$; Amdahl speedup = $1/((1-f) + f/s)$; pipeline speedup = $nk/(k+n-1)$; cache: AMAT = hit + miss rate $\times$ penalty; EAT with page faults = $(1-p)m + p \times$ fault time.
- Deadlock-free if resources $\geq n(m-1)+1$; RR/SJF waiting = turnaround - burst; disk FCFS, SSTF, SCAN, C-SCAN.
- Precision = $TP/(TP+FP)$; recall = $TP/(TP+FN)$; $F1 = 2PR/(P+R)$; entropy $H = -\sum p \log_2 p$; gain = $H(\text{before}) - \text{weighted } H(\text{after})$; gradient step $w = w - \eta \times \text{grad}$.

Appendix C: Sources and Where to Practise More

- Free past-paper collections consulted (public listings): GitHub repository golammostafa13/buet-msc-cse-questions (papers for 2017, 2018, 2019 and 2024 with solutions); the 2021 online exam MCQs listed on itjobexam.com and Scribd; a public listing of the May 2019 CSE paper; RakibHasanFahim/BUET-CSE-MSc-Admission (650-question original practice bank).

- The two Scribd documents you linked could not be opened by my tools (they require JavaScript). The "Admission Help Guide" is a commercial book, so its content is not reproduced here. If you have it, compare its solved papers against Appendix A.
- Recommended textbooks: CLRS (algorithms), Silberschatz (database, OS), Kurose and Ross (networks), Patterson and Hennessy (architecture), Hopcroft-Motwani-Ullman (automata), Dragon Book (compilers), Rosen (discrete mathematics).